UPTEC XX XXXXX

Degree project 30 credits

August, 2026

# Heterogeneous Multi-Robot Collaboration Using Language Models

## Exploring Model Choice, Planning Architecture, and Prompt Configuration

Rohit Joseph Mamutil

Master's Programme in Embedded Systems

Heterogeneous Multi-Robot Collaboration Using Language Models

Rohit Joseph Mamutil

## Abstract

Heterogeneous warehouse fleets combine agents with complementary roles, such as carrier vehicles and pickers, whose decisions must be coordinated under task dependencies, shared resources, and battery constraints. This thesis investigates whether locally deployed, off-the-shelf large language models (LLMs) can support high-level decision-making for this form of multi-robot collaboration. A grounded control loop was implemented in the Battery-TA-RWARE simulation environment: the current warehouse state is translated into a prompt, the model selects discrete macro-actions, and proposed actions are validated against environment-defined feasibility constraints before execution.

The experiments evaluate six language models spanning approximately 1B to 14B parameters across six warehouse scenarios. They compare centralized joint planning with per-agent shared-context planning, natural-language with JavaScript Object Notation (JSON)-based state representation, and three prompted objective families addressing shelf delivery, LLM-call usage, and battery management. Performance is assessed using shelf deliveries, model-call frequency, a call-normalised delivery measure, and charging behaviour. Generation and action-validation failures are logged for diagnostic run validation.

The results show that planning architecture, prompt configuration, model choice, and scenario configuration all influenced collaborative task performance. The natural-language configuration and shared-context planning were generally associated with stronger delivery performance in the evaluated setting, although the latter required more model interactions. Across the selected model artefacts, performance did not increase consistently with nominal parameter size, and changing the stated objective produced different behavioural responses across models.

These findings indicate that language models can produce grounded decisions within a constrained heterogeneous-robot control interface, while also showing that effective collaboration depends on the interaction between the model and the surrounding planning system. Given the limited number of runs and initialisations, the results are interpreted as descriptive evidence and motivate broader evaluation of language-model-based robot collaboration.

**Faculty of Science and Technology**

**Uppsala University, Place of publication Uppsala**

Supervisor: Xuezhi Niu  Subject reader: Didem Gürdür Broo

Examiner: Ramanathan Thinniyam Srinivasan

# Sammanfattning

Heterogena robotflottor i lager består av agenter med kompletterande roller, såsom transportfordon och plockrobotar, vars beslut måste samordnas med hänsyn till uppgiftsberoenden, gemensamma resurser och batteribegränsningar. Detta examensarbete undersöker om lokalt körda, allmänt tillgängliga large language models (LLMs) kan stödja beslutsfattande på hög nivå för denna form av samarbete mellan flera robotar. En grundad styrloop implementerades i simuleringsmiljön Battery-TA-RWARE: lagrets aktuella tillstånd omvandlas till en prompt, modellen väljer diskreta makrohandlingar och de föreslagna handlingarna valideras mot miljöns genomförbarhetsvillkor före exekvering.

Experimenten omfattar sex språkmodeller med ungefär 1 till 14 miljarder parametrar och sex lagerscenarier. De jämför centraliserad gemensam planering med delad kontext och separat planering för varje agent, representation i naturligt språk med JavaScript Object Notation (JSON)-baserad representation samt tre promptbaserade målfamiljer som behandlar hyllleveranser, användning av LLM-anrop och batterihantering. Prestandan bedöms med hjälp av bland annat antal hyllleveranser, anropsfrekvens, ett leveransmått normaliserat efter antal modellanrop och laddningsbeteende. Fel vid generering eller handlingsvalidering loggas för diagnostisk validering av körningarna.

Resultaten visar att planeringsarkitektur, promptkonfiguration, modellval och scenariokonfiguration påverkade samarbetets prestanda. Konfigurationen med naturligt språk och planeringsarkitekturen med delad kontext var generellt förknippade med bättre leveransresultat i den utvärderade miljön, även om den senare krävde fler modellanrop. Bland de utvalda modellartefakterna ökade prestandan inte konsekvent med den nominella parameterstorleken, och förändringar av det angivna målet gav olika beteenden hos olika modeller.

Resultaten indikerar att språkmodeller kan fatta grundade beslut inom ett begränsat styrgränssnitt för heterogena robotar, men också att ett effektivt samarbete beror på samspelet mellan modellen och det omgivande planeringssystemet. Eftersom antalet körningar och initialiseringar är begränsat tolkas resultaten som deskriptiva och motiverar bredare utvärderingar av språkmodellbaserat robotsamarbete.

# Acknowledgements

I would like to express my sincere gratitude to Professor Didem Gürdür Broo for providing the opportunity to undertake this thesis within the Cyber-Physical Systems Lab (CPS Lab) at Uppsala University, and to my supervisor, Xuezhi Niu, for his guidance, constructive feedback, and support throughout this research.

I am grateful to my examiner, Ramanathan Thinniyam Srinivasan, for his valuable comments and suggestions, which helped improve the quality of this thesis.

This thesis has been a challenging and rewarding experience. It required bringing together knowledge from multiple disciplines, including warehouse automation, heterogeneous multi-robot systems, and large language models. Combining these areas into a single research project demanded continuous learning and a willingness to explore unfamiliar domains.

The rapid development of large language models made this research particularly challenging. Keeping pace with the latest developments while integrating them into a grounded decision-making framework and evaluating them through extensive experiments required continuous learning, patience, and significant computational resources.

Throughout this journey, I would like to thank my family for their unwavering love, support, encouragement, and belief in me, which gave me the strength to keep going.

Finally, I would like to acknowledge the developers and maintainers of the open-source software, libraries, and simulation environments that made this research possible. Their contributions have been invaluable in enabling research such as this and continue to advance the broader scientific community.

# Declaration of Generative AI Use

OpenAI Codex was used to support code development, LaTeX editing, report structuring, and review of drafts for clarity, consistency, and possible omissions. OpenAI ChatGPT was used occasionally for rephrasing and suggestions concerning sections of the report. Generative AI was also used to create Figures 2 and 3, with Figure 2 subsequently edited manually. All research decisions, implementation choices, experimental work, data analysis, interpretation, and final content remain the responsibility of the author. Suggestions produced by these tools were critically reviewed and revised before inclusion. Neither tool was treated as a source of scientific evidence or used to generate experimental measurements.

# Contents

# Acronyms

**AGV** automated guided vehicle. 17, 18, 23–27, 29, 30, 32, 35

**JSON** JavaScript Object Notation. i, ii, 9, 10, 20, 21, 26, 28, 32, 33, 35, 37, 38, 41, 45–47, 52, 55

**LLM** large language model. i, ii, 10, 11, 14, 15, 19, 20, 22, 23, 27, 29, 31, 33, 35, 37, 42, 47, 50, 52, 55, 57, 68

**TA-RWARE** task-assignment robotic warehouse environment. 17

# 1 Introduction

Long before cities, agriculture, written records, or modern technology, humans lived in small groups navigating an uncertain world. Survival depended not only on individual intelligence but also on the ability to act collectively. Groups hunted, gathered food, protected offspring, shared resources, and learned from one another. Many accounts of human social evolution emphasise cooperation and social bonding as important pressures shaping communication [1, 2].

As human societies became larger and their activities more complex, the demands placed on communication also increased. Groups needed to remember where resources had previously been found, teach tool-making techniques, coordinate hunting strategies, divide responsibilities, and adapt plans when circumstances changed. Improved communication was needed to refer to distant locations, future actions, past experiences, hidden constraints, and the intentions of other individuals [3, 1].

Language likely emerged gradually alongside human social and cognitive evolution rather than appearing as a fully formed system [4]. Simple signals became richer, more structured, and more expressive. Sounds, gestures, and shared symbols acquired stable meanings that enabled more sophisticated interactions. As collective activities became more demanding, communication evolved to support them. At the same time, social and organisational pressures favoured further advances in communication [1, 2].

This co-evolution produced a feedback loop in which improved communication supported larger groups, specialised roles, more effective teaching, and advanced planning, while these social developments encouraged further refinements in communication. Many accounts of language evolution consequently view language and collective action as deeply intertwined phenomena. Language is often associated with shared intentionality, social organisation, cooperation, and the practical demands of acting together in complex environments [3, 1, 2, 4].

The development of written language further extended these capabilities. Knowledge, procedures, plans, and experiences could now persist beyond immediate interactions and be shared across larger groups and across generations [5]. Human societies accumulated large bodies of recorded information, creating new opportunities for learning, coordination, and collective problem-solving.

The growth of written language and, later, digital information created an unprecedented repository of human knowledge [5]. Books, articles, manuals, conversations, reports, and countless other forms of written communication captured facts, goals, plans, instructions, explanations, negotiations, and other artefacts of human problem-solving. As digital text collections expanded, researchers began developing computational methods

capable of modelling and generating language automatically [6].

Early computational language models represented language through statistical relationships between words and phrases [6]. Neural probabilistic language models later showed how such relationships could be learned through distributed representations [7], and word-embedding methods scaled distributed representations to large text corpora [8]. The introduction of transformer-based architectures marked a significant advance in language modelling [9]. Together, these developments contributed to the emergence of large-scale foundation models trained on broad data collections [10].

Modern large language models (LLMs) are trained on extensive collections of human-generated language [11, 10]. Unlike systems based on manually encoded rules or task-specific reasoning procedures, they acquire behaviour from statistical patterns in examples contained within their training data [11, 10]. This exposes the models to many forms of human activity expressed through language, including planning, instruction following, explanation, negotiation, coordination, and collaborative problem-solving [11, 10].

Although LLMs are fundamentally trained to predict the next token in a sequence, recent studies suggest that they can exhibit capabilities extending beyond simple text completion [10, 6]. Under suitable prompting strategies, language models have been shown to generate plausible plans [12], decompose complex tasks into intermediate sub-goals [13], reason over sequences of actions [14], and explore alternative reasoning paths when constraints or objectives change [15]. These observations have motivated growing interest in the use of language models as components of decision-making and planning systems rather than solely as tools for text generation.

A natural question follows: if language evolved alongside sophisticated forms of collective action, and modern language models learn patterns from descriptions of goals, plans, responsibilities, and constraints, could such models contribute to decision-making in situations where multiple entities must work together towards a shared objective? The present thesis investigates this possibility in the context of multi-agent systems.

Many real-world problems cannot be solved effectively by a single decision-making entity. Instead, they involve multiple entities that must operate within a shared environment while pursuing individual or collective objectives. In artificial intelligence and robotics, such entities are commonly referred to as agents. An agent is a system that perceives its environment and selects actions based on those perceptions [16]. When multiple agents operate within the same environment, the resulting system is commonly referred to as a multi-agent system (MAS) [17].

As the number of agents increases, decision-making becomes more challenging. Agents may compete for resources, interfere with one another, require access to shared infor-

mation, or depend on each other to complete tasks. Consequently, the ability to organise and align the actions of multiple agents becomes a central concern in multi-agent and multi-robot systems [18].

Researchers have proposed several concepts to describe different forms of collective behaviour among agents. Terms such as coordination, cooperation, and collaboration are often used throughout the literature, although their meanings are not always consistent. In some cases the terms are used interchangeably, while in others they describe distinct levels of interaction between agents. Because these concepts are central to the present thesis, the following definitions are adopted.

In this thesis, *coordination* refers to arranging actions, timing, information, or resource usage so that agents do not interfere with one another and can make progress towards their objectives. *Cooperation* refers to situations in which agents pursue a shared objective, even if their individual actions are not tightly interdependent. *Collaboration* is used for the stronger case where agents depend on one another to complete a task and where progress requires explicit interaction between different roles or capabilities. These distinctions are important because the literature does not always use the terms consistently, particularly across multi-agent systems, multi-robot systems, and human teamwork [18].

A particularly important class of multi-agent systems is the multi-robot system (MRS), where the agents are physical robots operating within a shared environment. Multi-robot systems have been studied in domains such as manufacturing, logistics, exploration, transportation, and search-and-rescue operations [18]. As in human teams, effective performance often depends on the capabilities of individual agents and their ability to coordinate and collaborate with one another.

Multi-robot systems appear in many domains, including manufacturing, transportation, agriculture, exploration, and logistics. Among these, warehouse logistics has received particular attention due to the rapid growth of e-commerce and the increasing demand for fast order fulfilment. Global e-commerce continues to expand and constitutes an increasingly important component of the digital economy, placing substantial pressure on warehouse operations, fulfilment networks, and inventory management systems [19]. As a result, improving warehouse efficiency has become both an important industrial objective and an active area of research. Warehouse environments are also particularly relevant from a multi-agent systems perspective. Fulfilling customer orders requires the interaction of multiple resources, including storage locations, transportation systems, workstations, inventory-management processes, and, increasingly, autonomous robots. These resources operate under shared constraints and must be coordinated to ensure efficient order fulfilment, making warehouse logistics a practical and representative domain for studying multi-agent and multi-robot decision-making [20].

Consider a customer placing an online order. The requested item may be stored on a shelf among thousands of products distributed throughout a warehouse. To fulfil the order, the item must be located, retrieved from storage, transported to a picking or packing area, prepared for shipment, and eventually dispatched for delivery. Throughout this process, multiple resources must interact efficiently to ensure that the correct item reaches the customer on time [20].

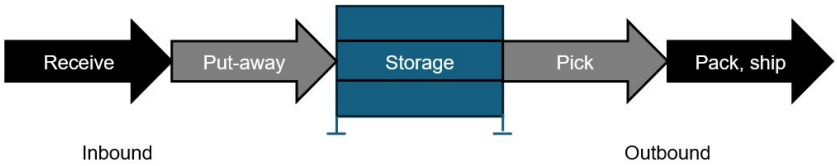


**Figure 1** Overview of the warehouse goods flow.

Warehousing is a broad discipline with its own terminology and operational practices. This thesis introduces only the concepts necessary to understand the experimental environment; a more comprehensive discussion of warehouse operations can be found in Bartholdi and Hackman [20].

In a typical warehouse, products are stored in designated storage locations such as shelves, racks, bins, or movable storage units. Customer requests are represented as orders that specify which items must be retrieved. The process of retrieving requested items from storage is commonly referred to as picking. Depending on the warehouse design, picked items may be transported to packing stations, where orders are prepared for shipment, before being sent to delivery or dispatch stations for onward transportation [20, 21].

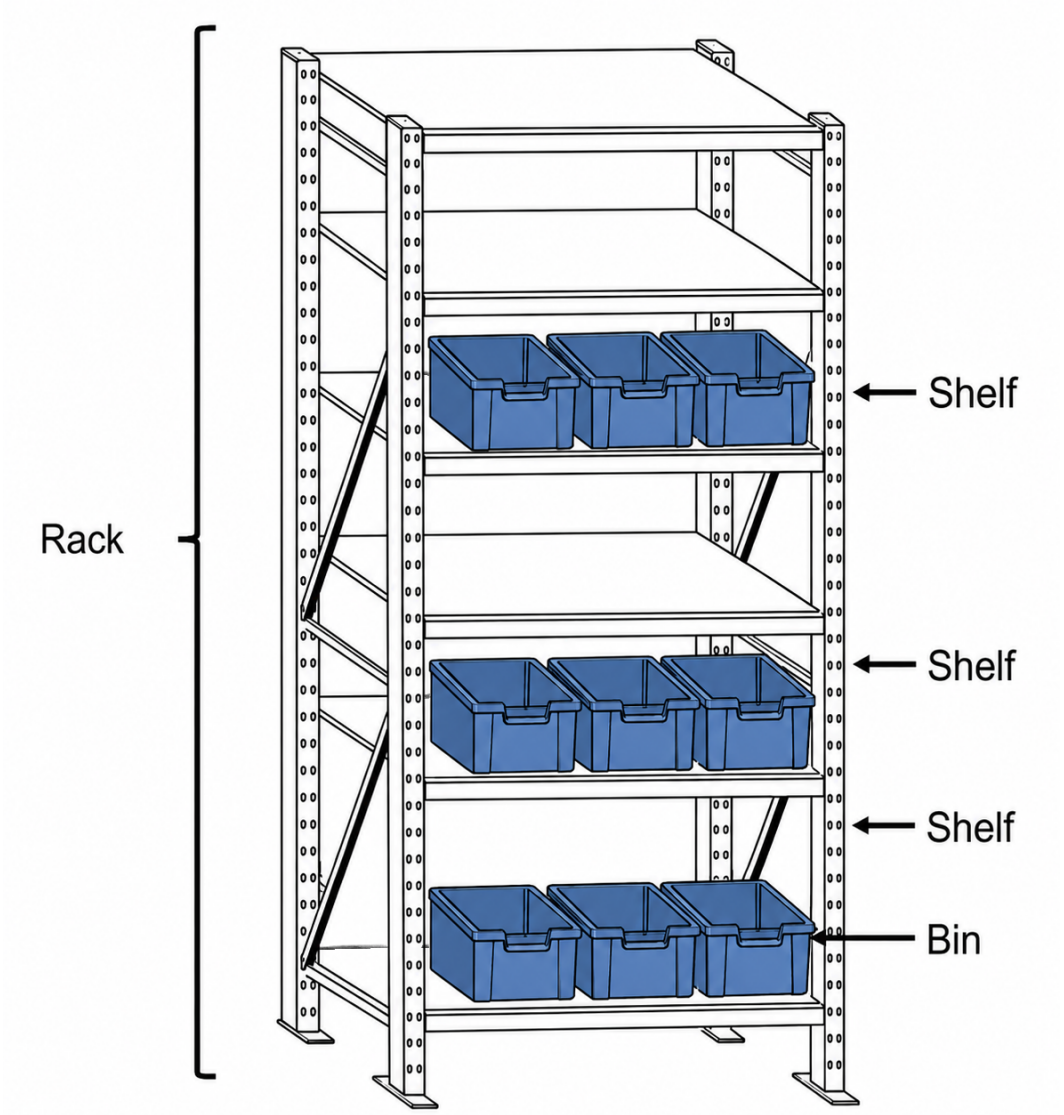


**Figure 2** Example warehouse storage hierarchy showing a rack composed of multiple shelves and storage bins. *AI-generated and subsequently manually edited.*

The resources involved in these activities may include human workers, automated storage systems, mobile robots, workstations, and warehouse management software. Together, these components form a workflow whose objective is to fulfil customer orders accurately and efficiently. Traditionally, warehouses operated according to a person-to-goods paradigm. Human workers travelled through storage aisles to locate and retrieve products. While straightforward to implement, worker travel often represented a signif-

icant portion of fulfilment time and operational cost [21].

As order volumes increased, particularly with the growth of e-commerce, alternative approaches emerged. In goods-to-person systems, products or storage units are brought to stationary workers instead of requiring workers to travel throughout the warehouse. This reduces travel time but introduces new coordination challenges, since storage units, workstations, and transportation resources must now be scheduled and synchronised effectively [22, 23].

Robotic warehouse systems emerged as a natural extension of this evolution. A well-known example is the Kiva-style warehouse system, in which fleets of mobile robots transport inventory pods between storage areas and stationary workstations [24]. Unlike traditional racks, which remain fixed in place, inventory pods are movable storage units containing multiple shelves and stored products. Mobile robots navigate beneath these pods, lift them, and transport them to workstations where items can be retrieved [24].

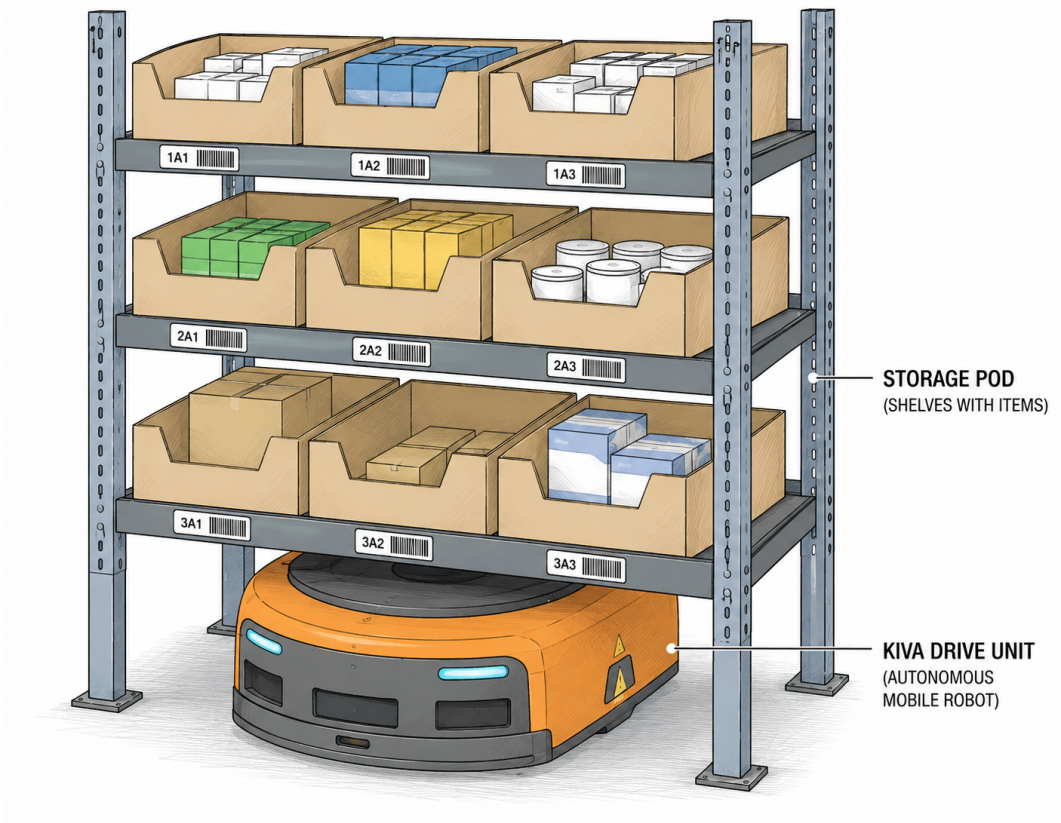


**Figure 3** Illustration of a Kiva-style inventory pod. The pod contains multiple shelves used for product storage, while a mobile drive unit operates beneath the pod to transport it throughout the warehouse [24]. *AI-generated from reference images.*

Rather than asking workers to travel to products, the warehouse brings products to workers. By transporting storage units to stationary workstations, goods-to-person (GTP) systems reduce the need for worker travel and can improve fulfilment efficiency [24]. At the same time, they introduce additional coordination requirements among robots, storage units, and workstations. The challenge is no longer limited to locating products and fulfilling orders. The warehouse must now determine which storage unit should be moved, which robot should perform the movement, which workstation should receive the requested items, and how shared resources should be allocated when multiple requests compete for attention. As the number of robots, workstations, and orders increases, these decisions become increasingly interdependent [24, 23].

At this point, warehouse operation becomes a coordination problem. Robots may compete for pathways, storage units, charging stations, or workstations. Tasks may depend on the completion of other tasks. Some activities can be performed independently, while others require multiple resources or specialised roles to interact successfully. The overall performance of the warehouse depends on the capabilities of individual agents and the effectiveness of their coordination [18]. Although warehouse logistics provides a useful example, similar challenges appear throughout multi-agent systems. Autonomous vehicle fleets, drone swarms, manufacturing systems, search-and-rescue teams, and service robots must all operate under shared constraints while pursuing common objectives. The underlying challenge is therefore more general than warehouse automation: how can multiple agents make effective decisions while acting within a shared environment?

While coordination is essential in robotic warehouses, many real-world multi-agent systems require a stronger form of interaction. Coordination may be sufficient when agents perform largely independent tasks while sharing resources. Collaboration, however, requires agents to depend on one another in order to achieve a common objective. In such settings, successful task completion depends on the interaction of agents with complementary roles or capabilities [18, 25].

Collaborative decision-making introduces additional challenges. Decisions made by one agent may influence the opportunities available to others, while task completion may depend on actions performed by multiple agents. Effective behaviour therefore requires consideration of shared goals, task dependencies, resource constraints, and the expected behaviour of teammates [25]. These characteristics make collaborative multi-agent systems an interesting domain for investigating whether language-model-based planning and decision-making can provide useful support.

In many practical multi-agent systems, this interdependence arises because agents possess complementary capabilities rather than identical roles. Progress may therefore depend on interactions between specialised agents that must work together to complete

tasks.

Evaluating such decision-making approaches directly in physical robotic systems can be costly, difficult to reproduce, and potentially disruptive to ongoing operations. Changes to coordination or collaboration strategies may affect safety, productivity, equipment utilisation, and interactions between agents operating within the environment. Consequently, simulation has become a widely adopted methodology for studying multi-agent and multi-robot systems [26, 27].

Simulation environments provide controlled and repeatable conditions in which task distributions, agent capabilities, resource constraints, and environmental characteristics can be varied systematically. This enables researchers to compare different approaches under identical conditions before considering deployment in physical systems.

The growing adoption of simulation has led to the development of standardised benchmark environments. Frameworks such as OpenAI Gym and its successor Gymnasium provide common interfaces that support reproducible experimentation across a wide range of reinforcement-learning and decision-making problems [28, 29]. Warehouse-inspired benchmark environments have become particularly popular because they capture many of the coordination and collaboration challenges encountered in real-world systems. Agents must share resources, resolve conflicts, allocate tasks, and operate towards common objectives while acting under environmental constraints. Although simplified relative to real warehouses, such environments provide a useful platform for studying multi-agent decision-making [27].

Many approaches have been proposed for decision-making in multi-agent and multi-robot systems. Depending on the application, researchers have employed optimisation methods, heuristic approaches, rule-based control, task-allocation algorithms, and more recently multi-agent reinforcement learning. These techniques have demonstrated success across a variety of coordination and planning problems, but they may require domain-specific design, extensive training, reward engineering, or adaptation when objectives and environmental conditions change [30, 31, 26].

Recent advances in large language models have motivated interest in their use beyond traditional text-generation tasks. Their demonstrated ability to generate plans, reason about constraints, decompose tasks, and interpret structured descriptions of environments has led researchers to explore their use in planning and robotics applications [12, 32, 33]. These developments raise the possibility that language models could serve as high-level decision-making components in collaborative multi-agent systems.

Despite growing interest in language-model-based planning, relatively little work has investigated their use for collaborative decision-making among heterogeneous agents operating under shared constraints. Existing studies often focus on single-agent tasks,

embodied planning for individual robots, or coordination settings in which agents perform largely independent activities [12, 32, 34]. Less attention has been given to environments where successful task completion depends on interactions between specialised roles and where decisions must remain grounded in a constrained action space.

Several practical questions therefore remain open. How does collaborative performance vary among locally deployable language models spanning different parameter scales? How should planning responsibilities be distributed when multiple agents require decisions? Does the representation used to describe the environment influence the quality of the resulting decisions? Addressing these questions is important for understanding whether language models can contribute to collaborative multi-agent systems and how such systems should be designed and deployed in practice.

To investigate these questions, the present thesis evaluates language models as high-level decision-making components within a collaborative multi-agent setting. Rather than focusing on low-level robot control, the study examines how different design choices influence collaborative task performance. In particular, three factors are considered: model choice across nominal parameter scales, the architecture used for planning, and the representation used to describe the environment state.

One important design consideration concerns the planning architecture. In a *centralized planning* approach, a single prompt receives a shared description of the environment and returns decisions for all agents requiring planning. An alternative approach is *shared-context per-agent planning*, where agents are planned individually but each prompt still contains a summary of the shared environment state. These approaches represent different ways of distributing decision-making responsibility while maintaining access to global context.

Another important consideration concerns how information is presented to the language model. Software and robotic systems commonly exchange information using structured data representations rather than natural language. One widely adopted format is JavaScript Object Notation (JavaScript Object Notation (JSON)), which represents system state through structured key-value pairs and hierarchical data structures. Such representations are convenient for software integration because they can often be generated directly from program variables with minimal transformation.

However, language models are primarily trained on human-generated text and may therefore process information differently depending on how information is represented. Recent studies suggest that prompt structure and information representation can influence language-model behaviour and task performance, while structured formats may affect reasoning performance in some settings [35, 36]. This creates an important practical question for engineering applications. Should system state be provided directly in

a structured format such as JSON, or is it beneficial to invest additional effort in transforming the same information into natural-language descriptions that more closely resemble the data on which language models were trained? The present thesis investigates this question alongside model choice across parameter scales and planning architecture.

## 1.1 Research Questions

The thesis is guided by the following main research question:

> **Main Research Question:** Can prompted large language models (LLMs) support grounded high-level decision-making for heterogeneous multi-robot collaboration in a constrained warehouse environment?

To investigate this question systematically, the study examines three design factors: model choice across nominal parameter scales, planning architecture, and prompt configuration. The following sub-questions are considered:

1. **RQ1 (Model Choice Across Parameter Scales):** How does grounded collaborative task performance vary among selected locally deployable language models spanning different nominal parameter sizes?
2. **RQ2 (Planning Architecture):** How do centralized planning and shared-context per-agent planning differ in their ability to support collaborative task performance across scenarios with varying collaboration demands?
3. **RQ3 (Prompt Configuration):** How does collaborative task performance differ between the tested natural-language and JSON-based prompt configurations?

The research questions are evaluated in a warehouse-inspired collaborative environment involving heterogeneous agents operating under shared constraints. Performance is assessed using delivery count, model-interaction frequency, and behaviour across scenario configurations. Generation and action-validation failures are retained as diagnostic fields for run validation rather than treated as separate research outcomes.

## 1.2 Contributions

This thesis contributes:

- A grounded evaluation framework for studying language-model-based high-level decision-making in collaborative heterogeneous multi-robot systems. The approach constrains language-model outputs to environment-defined actions and validates proposed decisions against explicit feasibility constraints.
- Comparative empirical observations across selected models spanning different parameter scales, planning architectures, and prompt configurations in language-model-based collaborative decision-making. The results characterise differences in delivery performance and language-model utilisation under the grounded control interface.

## 1.3 Report Outline

Section 2 provides background on multi-robot coordination, warehouse task assignment, and LLM-based planning. Section 3 describes the environment, the proposed LLM control loop, the experimental design, and the main delimitations of the study. Section 4 presents the experimental results together with their interpretation. Section 5 discusses ethical and sustainability considerations. Finally, Section 6 concludes the thesis and outlines future work.

# 2 Background and Related Work

This chapter reviews the literature relevant to language-model-based task planning in heterogeneous multi-robot systems. It examines the evolution of task planning approaches, recent advances in language-model-based planning, grounding, environment representation, language-model selection, and simulation environments. Together, these developments establish the current state of the field and motivate the research presented in this thesis.

Task planning is a fundamental problem in multi-robot systems, concerned with allocating, ordering, and coordinating tasks among multiple robots to achieve a shared objective efficiently. Unlike single-robot planning, multi-robot systems must additionally consider interactions between agents, including resource sharing, communication, task dependencies, and conflicting objectives. As the number of robots and the complexity of the environment increase, the planning problem becomes increasingly difficult due to the combinatorial growth of possible task allocations and execution sequences.

Early research addressed these challenges using explicitly engineered planning and coordination strategies based on optimisation, heuristics, and predefined decision rules. Gerkey and Matarić [30] formalised the Multi-Robot Task Allocation (MRTA) problem and proposed a taxonomy for categorising task allocation methods according to robot capabilities, task characteristics, and allocation strategies. Their work established a common framework for analysing and comparing multi-robot planning algorithms and continues to influence subsequent research.

Yan *et al.* [18] subsequently reviewed the broader field of multi-robot coordination, covering auction-based task allocation, cooperative transport, communication-aware coordination, motion planning, and early learning-based methods. Representative systems such as MURDOCH demonstrated that distributed auction mechanisms could efficiently allocate tasks among heterogeneous robots, while other approaches incorporated temporal dependencies, spatial constraints, and robot availability during cooperative task execution. These studies showed that successful multi-robot task planning depends on task allocation, communication, coordination, synchronisation, and conflict resolution.

Although heuristic and optimisation-based approaches proved effective in structured environments, they relied heavily on handcrafted rules, explicit environment models, and manually designed objective functions. Adapting these systems to new environments or operational requirements often required substantial redesign, which encouraged increasing interest in learning-based approaches.

Learning-based planning, particularly Reinforcement Learning (RL) and Multi-Agent Reinforcement Learning (MARL), enables robots to learn coordination policies through

repeated interaction with the environment instead of relying on predefined decision rules. Compared with classical planning methods, MARL offers greater adaptability to dynamic environments by allowing cooperative behaviours to emerge through experience. It has therefore become a prominent approach for heterogeneous multi-robot coordination.

Papoudakis *et al.* [26] addressed the lack of standardised evaluation in MARL by comparing representative independent learning, policy-gradient, and value-decomposition algorithms across a diverse collection of cooperative tasks. Their study showed that algorithm performance depends strongly on task characteristics such as reward sparsity and coordination complexity while introducing the EPyMARL framework and additional benchmark environments to support reproducible evaluation.

Later work by Krnjaic *et al.* [27] applied hierarchical MARL to heterogeneous warehouse logistics involving autonomous mobile robots and human workers. Their manager-worker architecture decomposed decision-making into hierarchical levels, improving sample efficiency and warehouse throughput compared with conventional MARL methods and industrial heuristic approaches. The authors also introduced the Task Assignment Robotic Warehouse (TA-RWARE) benchmark, extending RWARE to support heterogeneous task allocation and providing a reproducible platform for evaluating high-level coordination strategies.

Subsequent research shifted attention towards improving cooperation through reward design. Niu and Gürdür Broo [37] proposed a biologically inspired reward shaping framework based on ecological symbiosis, demonstrating improved convergence stability and cooperative behaviour in heterogeneous robotic systems. Niu *et al.* [38] later incorporated symbiotic principles into a Centralised Training with Decentralised Execution (CTDE) MARL framework, enabling robots to coordinate battery management and workload distribution through shared state information. Their results demonstrated improvements in task performance, resource utilisation, and energy management in simulated warehouse environments.

Despite these advances, learning-based approaches remain dependent on environment-specific training, carefully designed reward functions, and computationally intensive policy optimisation. Adapting trained policies to new environments or task configurations often requires additional training or redesign. These limitations have led researchers to investigate pretrained large language models (LLMs) as an alternative paradigm for high-level task planning, seeking to exploit the reasoning capabilities acquired during large-scale pretraining instead of learning task-specific policies from scratch.

Li et al. [39] classify LLM applications in robotics according to the level of decision-making they support, including high-level task planning and task allocation, mid-level

motion planning, and low-level action generation. High-level planning concerns reasoning over task objectives and coordination, whereas lower levels focus on trajectory generation or direct robot control.

Existing LLM-based planning frameworks commonly employ language models as high-level decision makers while retaining conventional perception, navigation, and control modules. SayCan grounds language-model action selection with affordance estimates so that plans are biased towards actions that are both useful for the instruction and feasible in the current robotic state [32]. Inner Monologue extends this pattern by incorporating feedback from the environment into language-model planning, while ProgPrompt represents situated task plans as program-like structures over available robot actions [40, 41]. Code as Policies instead uses the language model to generate executable programs over predefined robot APIs, making code the intermediate policy representation between language instructions and embodied control [33]. Related zero-shot planning work shows that language models can propose action sequences from textual task descriptions when they are given an explicit action interface, although such plans still require grounding, validation, and replanning when deployed in physical or simulated environments [12].

Extending LLMs from single-robot planning to multi-robot systems introduces additional challenges related to communication, coordination, and information sharing among multiple autonomous agents. Unlike single-robot systems, where a single planning context is sufficient, multi-robot systems must determine how planning responsibilities and information should be distributed across multiple agents. Consequently, recent research has explored a range of planning and communication architectures for integrating LLMs into collaborative multi-robot systems [39].

One line of research investigates how individual language models should be orchestrated across multiple agents. Liu *et al.* [42] analysed several architectures for LLM-augmented autonomous agents, ranging from simple zero-shot planning to iterative self-reflection frameworks that incorporate previous observations and actions into subsequent reasoning steps. Their work also proposed BOLAA, an orchestration strategy in which a controller manages communication among multiple specialised language-model agents. Although their study primarily considers general autonomous-agent benchmarks rather than physical robot teams, the proposed orchestration strategies provide useful insights for multi-robot coordination.

In robotics, Mandi *et al.* [34] proposed RoCo, a multi-robot collaboration framework in which language-model agents discuss task strategies, generate sub-task plans, and revise plans using feedback such as collision checking. This work demonstrates how dialogue and shared feedback can be used to coordinate multiple robots, but it also illustrates the need for explicit grounding and validation when language models are used inside

embodied planning loops.

Although LLMs possess extensive world knowledge acquired during pretraining, they do not have direct knowledge of the robot's current environment. Consequently, generated plans may reference unavailable objects, infeasible actions, or incorrect assumptions about the world state, a phenomenon commonly referred to as hallucination. Grounding addresses this problem by constraining language-model reasoning using information obtained from the environment, ensuring that generated plans remain both logically and operationally feasible.

Existing work has explored several approaches for grounding LLMs in robotic task planning. SayCan constrains language-model reasoning through affordance-based action selection, ensuring that generated actions are executable in the current environment [32]. Feedback-based approaches further improve planning reliability by incorporating observations from the environment during execution, allowing the language model to identify infeasible plans and refine subsequent decisions through iterative replanning [40, 43]. Recent surveys identify grounding as a fundamental requirement for improving the robustness and reliability of LLM-based robotic planning systems [39].

Regardless of the grounding mechanism employed, the language model ultimately reasons over an abstract representation of the environment describing the available objects, actions, and current system state. Existing work has represented this information using natural-language descriptions, symbolic planning languages, executable code, and other structured representations. The choice of representation influences both the information available to the language model and the ease with which generated plans can be validated and executed, making environment representation an important design consideration for LLM-based task planning.

Structured representations such as JSON have become increasingly common in LLM-based planning because they encode hierarchical information through key-value pairs while remaining machine-readable and readily integrated with existing robotic software. In contrast, natural-language descriptions provide greater flexibility in expressing task context and relationships between entities. Consequently, both structured and natural-language representations have been adopted for conveying environment state to language models. However, recent work suggests that imposing restrictive formats may also influence language-model reasoning. Tam *et al.* [36] showed that strict output format constraints can reduce LLM performance on reasoning tasks, highlighting a potential trade-off between structured representations and generation flexibility. Despite the increasing use of both approaches, their impact on high-level task planning performance in heterogeneous multi-robot systems remains insufficiently understood.

Language model selection represents another important design consideration in LLM-

based robotic planning. In applied robotics and embedded systems, constraints on computation, memory, energy consumption, and inference latency may favour the deployment of smaller language models over larger alternatives [44]. However, smaller models often exhibit weaker reasoning capabilities and may therefore require more carefully designed prompts, structured task representations, or additional validation mechanisms to achieve reliable planning performance. Xiao et al. and Mandi et al. [45, 34] investigated prompt design as a means of compensating for limited model capacity and improving decision quality in resource-constrained deployments. At the same time, studies have shown that imposing restrictive prompt or output formats may negatively influence language-model reasoning, suggesting a trade-off between structured machine-readable representations and generation flexibility [35, 36]. Collectively, these studies indicate that planning performance depends not only on the choice of language model, but also on how planning problems are represented and communicated to the model.

A variety of simulation environments have been developed to evaluate multi-robot planning and coordination algorithms. These include warehouse-specific benchmarks, industrial warehouse simulators, and general-purpose robotic simulation platforms such as NVIDIA Isaac Sim, with the choice of simulator typically depending on the planning problem under investigation [27, 38].

Among warehouse-specific benchmarks, the Multi-Robot Warehouse (RWARE) environment introduced by Papoudakis *et al.* [26] models a partially observable grid-world warehouse in which multiple robots cooperate to retrieve requested shelves and deliver them to workstations. Agents receive sparse rewards only after successfully completing a sequence of coordinated actions, making coordination difficult and providing a challenging benchmark for cooperative multi-agent reinforcement learning.

To investigate heterogeneous task allocation, Krnjaic *et al.* [27] extended RWARE to develop the Task-Assignment Robotic Warehouse (TA-RWARE) environment. TA-RWARE introduces two heterogeneous agent roles, Automated Guided Vehicles (AGVs) and picker robots, which cooperate to complete warehouse operations. Instead of directly controlling robot motion, agents select target locations while navigation between locations is performed using a predefined path-planning heuristic. By abstracting low-level navigation, the environment enables researchers to focus on high-level task assignment and coordination in heterogeneous multi-robot systems.

# 3 Methodology

## 3.1 Overview

This chapter describes the methodology used to evaluate language models as high-level planners in the Battery-TA-RWARE warehouse simulator. The evaluation focuses on collaborative decision-making for heterogeneous warehouse robots. The evaluated controller operates at the task-assignment level; low-level robot control, navigation, and motion planning are handled by the simulator.

The study adopts a controlled comparative experimental design. The simulator, warehouse scenarios, action interface, execution pipeline, and evaluation metrics remain unchanged across the reported experiments. The comparisons examine performance differences among the selected model artefacts spanning different nominal parameter scales, planning architectures, prompt configurations, and objective configurations.

The methodology begins with the Battery-TA-RWARE environment, followed by the language-model control pipeline, planning architectures, prompt representations, experimental scenarios, evaluation metrics, and experimental protocol.

## 3.2 Environment: Battery-TA-RWARE

Battery-TA-RWARE is a discrete-time, grid-based warehouse simulation derived from task-assignment robotic warehouse environment (TA-RWARE) [27]. At any time, a set of shelves is requested. Carrier automated guided vehicles (AGVs) must transport requested shelves to goal locations and then return them to storage. Picker agents are required to meet AGVs at shelf locations for load and unload interactions, creating explicit heterogeneous coordination constraints. Agents additionally have a battery state that decreases with movement and interactions; charging stations are available at fixed locations.

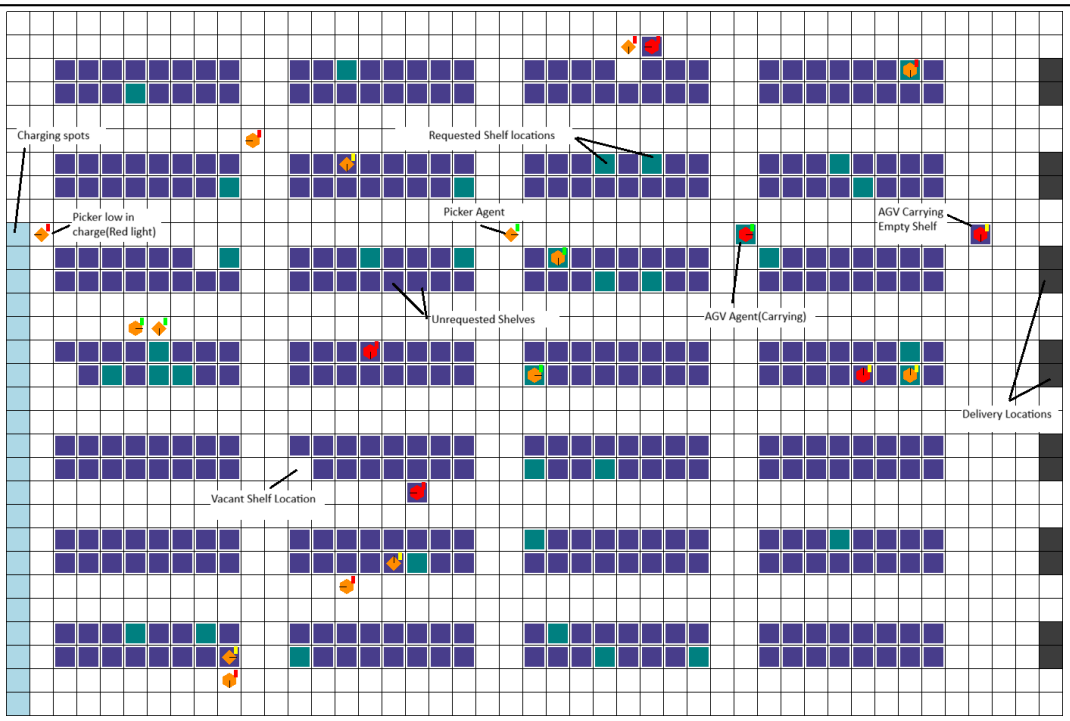


**Figure 4** Illustration of the task-assignment warehouse setting used in this thesis.

The environment exposes a discrete action space where each action corresponds to selecting a target location:

- **No-op action:** keep the agent idle or waiting for the current step.
- **Shelf actions:** choose a shelf location (e.g., to pick up or return a shelf).
- **Goal actions:** choose a delivery station (primarily relevant when an AGV carries a requested shelf).
- **Charging actions:** choose a charging station to recover battery.

Low-level motion toward the selected target is handled internally by the environment (A* path planning and collision resolution), while the controller is evaluated on the quality of these high-level decisions.

The simulated warehouse layout is generated from the environment id. The size keyword determines the number of shelf rows and shelf columns: `tiny` uses one shelf row and three shelf columns, `small` uses two shelf rows and three shelf columns, and `medium` uses two shelf rows and five shelf columns. With the configured column height

of eight cells, these correspond to larger grid layouts as the size increases. Goal cells are placed along the lower part of the warehouse, charging stations are placed along the upper row, and the number of charging stations is limited to the number of agents in the scenario. The request queue size for the tiny, small, and medium configurations used here is 20 shelves, which also gives the maximum number of deliveries possible in one run when all requested shelves are completed.

Battery state is included in the simulator state and is logged during each run. Each agent is initialised with a battery level sampled between 20 and 100. Forward movement reduces battery by 1, and load or unload interactions reduce battery by 2. When an agent executes the charging action at a charging station, its battery increases by 5 per simulation step up to a maximum of 100. Battery values are clamped at zero. Episodes continue when an agent reaches zero battery. Battery depletion is represented in the logged battery levels and in the battery-related prompt context. The battery-aware experiments evaluate whether the LLM controller selects charging actions and avoids low-battery states under this simulator model.

## 3.3 Language-Model Control Loop

Figure 5 shows the main components of the experimental framework. The Experiment Runner coordinates the warehouse environment, prompt construction, language-model inference, response processing, and result logging. During each planning cycle, the current state is converted into a prompt, the generated actions are parsed and applied to the environment, and the resulting experiment data are recorded.

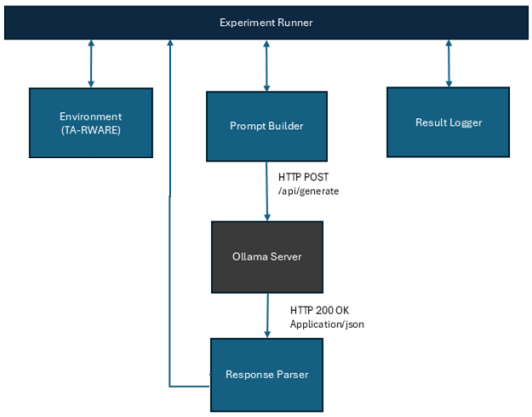


**Figure 5** Component-level architecture of the experimental framework and the primary control and data exchanges in the planning loop.

The LLM policy is implemented as a text-in / discrete-action-out controller:

1. **State summarisation:** extract salient features of the global state (requested shelves and their action IDs, per-agent position, target, battery, and carrying status, and charging occupancy).

2. **Candidate action set:** compute a valid-action mask from the environment and derive the prompt-visible candidate set from those feasible actions. These candidates are presented as compact role-relevant action rows with action IDs and guidance fields such as distance, tags, and charging-related instructions.

3. **Prompting:** generate either a shared-context prompt for one agent or a centralized prompt containing planning blocks for all agents being replanned. In both architectures, the selected prompt representation is either natural language or JSON. Both representations describe the relevant agent roles, current state, and explicit mappings from requested objects to action IDs.

4. **Querying:** send the constructed natural-language or JSON prompt to a local inference server (Ollama) and obtain a text response. For JSON prompts, the request uses the structured-output configuration described in Section 3.4.

5. **Parsing:** parse one or more integer action IDs, and optionally action hold durations, from the response. Natural-language responses are parsed from constrained text, whereas JSON responses are parsed from the required object structure.
6. **Validation:** validate each parsed action id against the current valid-action mask. If a proposed action is missing or invalid, the controller discards it and falls back to an idle action for that agent when available; in the action-persistence setting, an invalid persisted action clears the hold and forces a new query.
7. **Execution:** apply the validated action vector in the environment for one step, or hold accepted actions for multiple steps depending on configuration.

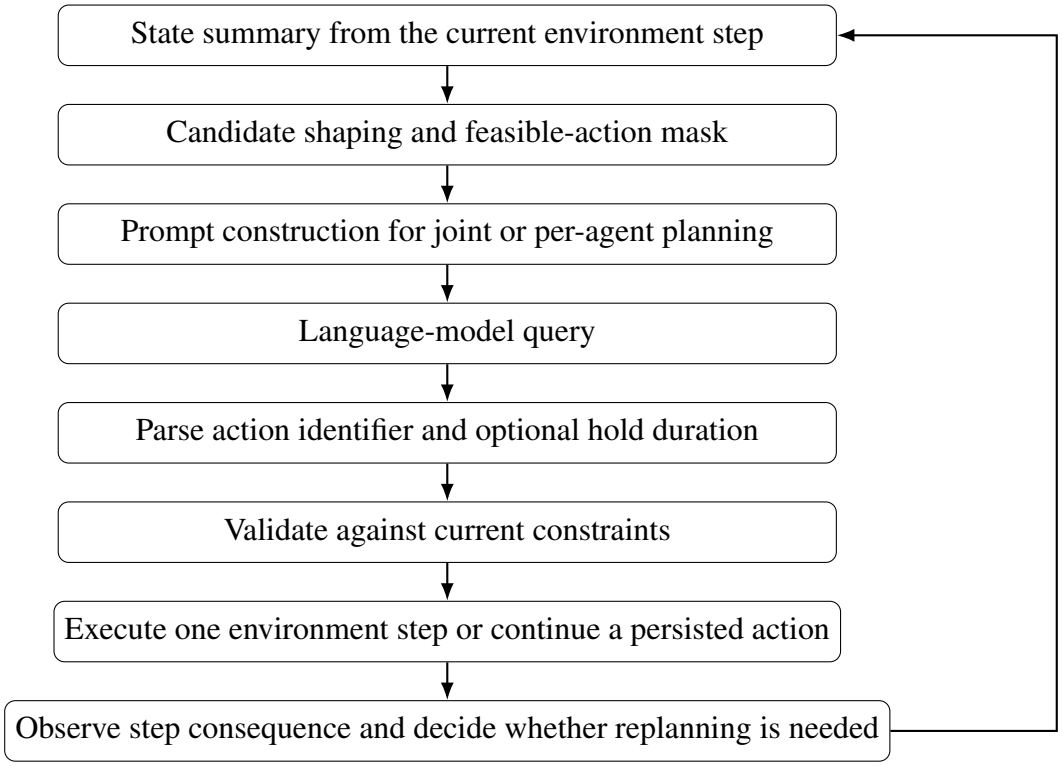


**Figure 6** Schematic of the language-model control loop used in the experiments.

The controller is a discrete-time, step-synchronous supervisory loop with conditional replanning. The environment advances one simulation step at a time, and at each step the controller reevaluates agent state and action validity. Agent queries are conditional, and the experiments use two related control schemes.

- **Step-synchronous replanning with environment-owned continuation:** agents that are currently available for planning may receive a high-level action, while

busy agents are not replanned and continue their current task until the environment releases them.

- **Step-synchronous replanning with action persistence and conditional override:** accepted actions may remain active for multiple future steps, so agents are not re-queried while the persisted action remains valid. Replanning is triggered when a persisted action expires, becomes invalid, or when the controller decides intervention is needed, including selected busy-agent cases.

The adopted design combines regular per-step observability with selective replanning. Regular state checks expose blocked progress, invalid persisted actions, expiring commitments, and changing support needs. Selective replanning limits unnecessary LLM calls. This matches a supervisory-control pattern in which the fleet is checked at regular intervals, ongoing tasks continue by default, and task-level decisions are issued when the current state requires intervention.

The control loop uses two grounding and validation mechanisms:

- **Grounding via action IDs:** the LLM output interface is restricted to discrete action IDs that are already defined by the environment.
- **Hard validation:** action masks enforce constraints such as invalid goals for picker agents and busy-state restrictions. Invalid LLM outputs are rejected before execution.

The reported behaviour reflects the complete control framework: prompt design, model output, candidate shaping, validation logic, and replanning policy act together inside the same supervisory loop.

## 3.4 Prompt Construction and Experiment Design

Prompt construction defines the interface between Battery-TA-RWARE and the language model. The evaluated models are used without task-specific fine-tuning, so the prompt specifies the task objective, state abstraction, action vocabulary, role constraints, and response format required for high-level action selection. This use of prompting follows the general prompt-engineering view summarised by Sahoo et al. [35], in which prompts supply task-specific instructions and context while model parameters remain unchanged. The taxonomy in that survey includes basic prompting methods, such as zero-shot and few-shot prompting, and more elaborate reasoning methods, such as

chain-of-thought prompting, ReAct, self-refinement, retrieval-augmented generation, tree-of-thoughts, and graph-of-thoughts. Within this taxonomy, the implemented planner uses manual zero-shot instruction prompting together with structured output constraints. Role-specific AGV and Picker information is used as task grounding inside the prompt. This prompt design supports the comparative focus of the study: selected models spanning different nominal parameter scales, coordination architecture, prompt configuration, and objective wording are evaluated under a controlled action-selection interface.

The experiments are organised as a controlled comparison within the grounded language-model control loop described above. Across the experiment families, the environment, discrete action space, validation layer, six-scenario set, local-inference setup, and metric extraction are kept consistent. The main factors that change are the model, coordination architecture, prompt configuration, scenario, and objective statement. This structure is used to compare how different ways of using the LLM affect collaborative task performance under comparable conditions.

**Simulator-to-language-model interface.** The experiment controller connects Battery-TA-RWARE to the language model through a text-in/discrete-action-out interface. At each planning step, the controller reads the current simulator state, computes the valid-action mask, constructs a prompt, submits the prompt as a single text field to a locally hosted Ollama server, parses the model response, validates the parsed action against the current mask, and then executes the resulting macro action in the environment. The language model selects integer action identifiers defined by the simulator. Path following, collision handling, battery updates, and load/unload mechanics are handled inside the environment.

The raw simulator state is translated into a symbolic prompt representation before it is shown to the model. Action identifiers are categorised as `NOOP`, `GOAL`, `CHARGING`, or `SHELF`. Requested shelves are rendered with shelf id, position, and corresponding action id. Agents are represented by role, position, current target, busy state, battery level, battery-need category, and carried-shelf state. Charging stations are represented by action id, position, and occupancy. For each candidate action, the prompt-visible row can include action id, action class, target position, estimated path distance, suggested hold duration, semantic tag, recommendation flag, and an instruction field indicating whether the action is currently allowed or should be avoided. Agent, shelf, and charging-station values are read from the current simulator state. Identifiers and hold durations are integers, positions are integer coordinate pairs, battery values and path distances are numeric, and busy, occupancy, and recommendation fields are Boolean. Battery-need categories, candidate rankings, suggested hold durations, semantic tags,

and conflict warnings are derived by the controller from the simulator state, valid-action mask, and path finder. This translation has two functions. It reduces the context required to describe the full grid state. It also exposes the high-level decision variables used by the model while leaving path-planning details to the simulator.

**Distance and candidate construction.** Distances shown in prompts are computed from the agent's current cell to the target action cell using the environment path finder. The same path-based distance is used to rank requested shelves, charging stations, support targets, and empty shelf-return candidates. If a path is unavailable, the candidate is either omitted by the valid-action/candidate logic or assigned a conservative fallback distance in the structured candidate representation. Candidate actions are not a verbatim copy of the full action space. They are selected from the valid-action mask and then shaped according to role and state. For an AGV, requested shelf actions are emphasised when it is not carrying a shelf, goal actions are emphasised when it carries a requested shelf, and empty shelf-return actions are emphasised after delivery when the carried shelf must be returned to storage. Charging actions are retained when battery state or control state makes charging relevant. For a Picker, the candidate set emphasises shelf actions that correspond to AGV support targets and charging actions when battery state makes charging relevant. This candidate shaping reduces prompt length, limits the number of infeasible or irrelevant alternatives, and provides the model with an action vocabulary aligned with the current warehouse requirement.

**Prompt structure.** The shared-context prompt is organised as a bounded action-selection problem. It begins with the objective statement and a concise environment description: the warehouse contains AGVs, Pickers, requested shelves, goal locations, charging stations, and battery constraints. The prompt then describes the normal collaboration flow: an AGV moves to a requested shelf, a Picker supports the shelf interaction, the AGV carries the shelf to a goal, and the delivered shelf is returned to an empty shelf location with Picker support. The current agent state, last selected action, remaining hold steps, movement warnings, blocking warnings, control state, role-specific context, hard constraints, candidate actions, and output contract are then appended. This ordering places general task semantics before state-dependent constraints, and places the deterministic output contract at the end where it is closest to generation.

The role-specific parts of the prompt encode robot capabilities and responsibilities. The AGV prompt states that an AGV can select requested shelf, goal, empty shelf-return, charging, or no-op actions depending on load state and battery state. It also states that an AGV should not leave a shelf interaction when Picker support is already inbound or required. The Picker prompt states that a Picker supports AGVs at shelf cells, that

support should occur at the same shelf action id as the AGV target, and that Picker support is not required at goal cells. These role-specific rules are included because heterogeneous coordination errors are otherwise common: an AGV may abandon a support shelf, or a Picker may move to an unrelated shelf or charge while support is pending.

The following compact example shows representative input fields and the same selected action in the two output configurations.

```
Representative input:
Current agent:
agent_0, role=AGV, position=(8, 3), battery=72, carrying=None

Candidate actions:
action=0, type=NOOP, distance=0
action=50, type=SHELF, distance=8
action=7, type=CHARGING, distance=11

Natural-language output:
Action: 50
Steps: 8
Reason: Move to the requested shelf.

JSON output:
{"reason":"Move to the requested shelf.","action":50,"steps":8}
```

Listing 1: Representative prompt input and corresponding natural-language and JSON outputs.

The complete programmatic prompt templates are retained in the centralized and shared-context experiment runners. The example above exposes the input fields and output forms relevant to the comparison, while Appendix A.3 provides a longer prompt excerpt.

| Prompt component | Information provided | Purpose |
|---|---|---|
| Objective statement | Delivery, call-aware, or battery-aware goal | Defines optimisation priority. |
| Environment description | AGVs, Pickers, shelves, goals, and chargers | Grounds the task domain. |
| Collaboration flow | Shelf pickup, Picker support, goal delivery, and shelf return | Encodes heterogeneous coordination. |
| Agent state | Role, position, battery, load, and busy state | Describes the current decision context. |
| Shared context | Requested shelves, charging occupancy, and support demands | Enables coordination across agents. |
| Candidate actions | Valid action IDs, action type, distance, and recommendation | Restricts the action space. |
| Conflict warnings | Blocking, cooldown, and unreachable targets | Prevents repeated poor choices. |
| Output contract | Action id, steps, reason, or JSON object | Enables deterministic parsing. |

**Table 1** Prompt components and their methodological purpose.

**Natural-language and JSON prompt variants.** Two prompt configurations are evaluated. Each configuration includes the representation of the simulator state, the required response format, and the associated parsing mechanism. The natural-language variant presents the same state information as headings and bullet-like textual sections and requests a compact response containing an action id, a hold duration, and a short reason. This representation is readable and close to the ordinary instruction-following format for language models. Its response is parsed from text, which makes format deviations possible.

The JSON variant presents the planning context as a structured object and requests a machine-readable object. For shared-context planning, the required response contains the keys `reason`, `action`, and `steps`. For centralized planning, the required response is a top-level object containing an `actions` list, with one action object for each planned agent. The permitted hold duration is 1–10 steps for AGVs and 1–3 steps for Pickers. Values outside the applicable range are restricted to that range during parsing. The Ollama request additionally uses structured-output mode for JSON prompts. This variant tests whether an explicit schema changes parseability and consistency while using the same state abstraction and candidate-action grounding as the natural-language prompt.

**Centralized and shared-context per-agent planning.** Two coordination architectures are compared. In the centralized architecture, one LLM call receives the global warehouse snapshot, committed actions for agents that are not being replanned, shared rules, role-specific rules, and one planning block for each agent requiring a new decision. The output must contain exactly one action for each listed agent and must not include actions for agents outside the planning blocks. This formulation exposes joint action selection across agents, including cases where AGV and Picker choices must be aligned. Its prompt size grows with the global context and the number of per-agent planning blocks, and its response is more complex than a single-agent decision.

In the shared-context per-agent architecture, the controller queries the model separately for each agent that requires replanning. Each per-agent prompt contains the local decision context for the current agent, including its role, position, load, battery level, busy state, and candidate actions. It also includes shared coordination information that can affect the local decision, such as requested shelves, charging occupancy, support demands, and battery-related context for other agents. The required output concerns only the current agent. This representation reduces output complexity and makes parsing simpler. Coordination is based on exposing a common symbolic context to each local decision. This architecture uses a single-agent output contract for each query.

| **Aspect** | **Centralized planner** | **Shared-context per-agent planner** |
|---|---|---|
| LLM calls | One call for all replanned agents | One call per replanned agent. |
| Prompt scope | Global state plus all planning blocks | Selected shared context plus current-agent state. |
| Output | Joint action list | Single-agent action. |
| Coordination mechanism | Explicit joint selection | Shared context across local decisions. |

Table 2: Comparison between centralized and shared-context per-agent planning architectures.

**Shared-context query order and decision timing.** In shared-context per-agent planning, eligible agents are processed using a fixed deterministic order based on current coordination and task state. AGVs requiring immediate Picker support are processed first, followed by Pickers when urgent support is pending, AGVs carrying shelves, agents below the battery threshold, and the remaining AGVs and Pickers; ties are resolved by agent id. This ordering determines the sequence of model queries but does not select the agents' actions. The physical environment state and valid-action masks do not change between per-agent queries. However, each accepted action is recorded as a

current commitment and may be included in the relevant shared information supplied to later prompts from the same simulation step. The completed action vector is executed only after all required decisions have been collected. The scheduling rule is part of the implemented shared-context configuration and was not evaluated as a separate experimental factor. The architecture results therefore compare the complete centralized and shared-context per-agent configurations as implemented.

**Output constraints and fallback behaviour.** The prompt constrains the response to the current action vocabulary. The model is instructed to choose only from the displayed candidate action IDs, to return an integer action ID, to return an integer `Steps` value within the allowed hold range, and to provide only a short reason. The prompt explicitly forbids action descriptions, coordinates, markdown, policy text, revised rules, recommendations, and additional commentary. These constraints are required because the simulator can only execute discrete action identifiers.

The runtime validation layer enforces these constraints independently of model compliance. If a response is missing, unparsable, outside the action range, or invalid under the current action mask, it is rejected and replaced by a fallback action, normally no-op when available. For JSON prompts, failed JSON parsing is logged separately; if possible, the parser recovers an explicit action field from malformed output, otherwise the output is treated as invalid. In a centralized response, an omitted planned agent is treated as a missing action for that agent and the fallback is applied. Actions outside the action range or invalid under the current mask are also replaced by the fallback action. Model requests use a 60-second HTTP timeout. HTTP 500 responses are retried up to five times with increasing delay. If a request still fails, times out, or returns no parseable valid action, the call is recorded as failed and the controller executes the fallback action, normally no-op, for one step. Accepted actions may be persisted for multiple steps through the `Steps` field. This persistence reduces unnecessary model calls, but a persisted action is cleared when it expires, becomes invalid, completes, or is associated with a recent resolution failure.

**Environment modifications supporting prompting and replanning.** Several environment and controller modifications were introduced to make the language-model control loop stable enough for comparative experiments. First, the control loop enables busy-state replanning. In the original macro-action setting, a busy agent continued its current assignment until the environment released it. The modified control loop can override selected busy assignments after a configured number of busy steps, while still allowing normal environment-owned continuation for ordinary progress and conflict resolution. This supports recovery from stale assignments while preserving se-

lective agent queries.

Second, Picker approach behaviour at shelf interactions was adjusted. Previously, a Picker could move into the shelf-interaction cell or stop immediately adjacent to it before the AGV arrived, which could block the AGV path and produce unstable outcomes unrelated to high-level assignment quality. The modified environment prevents a Picker from completing the final approach to a shelf-interaction target until the corresponding AGV is waiting there. Before that condition is met, the Picker waits approximately two path steps away. This keeps the support agent close enough for coordination while reducing premature occupation of the shelf-entry area.

Third, additional conflict-resolution state is exposed to the controller. Agents can be marked as resolving a conflict, recently failed on an unreachable target, or temporarily forbidden from selecting a target that repeatedly caused resolution failure. Blocking relationships are also summarised with reasons such as a Picker blocking an AGV path or an AGV waiting for Picker support at a shelf. These signals are inserted into the prompt as movement, blocking, and execution warnings. The prompt uses these signals to prevent repeated selection of the same blocked target and to make coordination-relevant states visible at the task-assignment level.

The underlying simulator retains responsibility for low-level navigation, A* path planning, collision handling, battery updates, and load/unload mechanics. The language-model interface, prompt-visible candidate construction, response parsing and validation, action persistence, busy-state replanning, Picker staging, and additional conflict information form the modifications evaluated in this thesis. The source implementation is available in the project repository[1] at commit `361499ed2498`. Because the Picker and replanning modifications affect execution dynamics, the reported results describe this version of the environment and are not treated as a direct comparison with previously published TA-RWARE results.

The systematically modified factors are model, prompt configuration, coordination architecture, scenario, and objective statement. The objective analysis uses three objective prompts: calls, battery plus shelf delivery, and battery. The calls objective asks the model to optimise LLM calls while still increasing shelf delivery. The battery plus shelf objective asks the model to maximise shelf deliveries while keeping agents charged enough to avoid critical battery levels. The battery objective asks the model to keep agents charged, prioritise charging before critical battery thresholds, and avoid battery depletion. Across these objective families, the controller and grounded action interface remain unchanged.

[1] https://github.com/Cyber-physical-Systems-Lab/polyphony

## 3.5 Scenarios, Models, and Metrics

The experiments use six scenario configurations for shared-context and centralized planning. These scenario descriptions are not additional prompt content; they are used for experiment design and logging, while the model receives the current warehouse state and candidate actions. The same scenario set is reused across objective families where possible so that changes in behaviour can be related to model, prompt, architecture, or objective differences with fewer confounds from task changes.

| **Scenario label** | **Warehouse and agents** | **Reason for inclusion** |
|---|---|---|
| tiny_balanced_1v1 | Tiny layout; 1 AGV, 1 Picker | Minimal heterogeneous case used to test whether the model follows the basic AGV-Picker collaboration loop. |
| small_balanced_2v2 | Small layout; 2 AGVs, 2 Pickers | Balanced multi-agent case with more concurrency than the tiny setting. |
| medium_balanced_4v4 | Medium layout; 4 AGVs, 4 Pickers | Larger balanced setting used to test scaling with more agents and longer paths. |
| small_agv_heavy_4v2 | Small layout; 4 AGVs, 2 Pickers | Tests whether limited Picker capacity creates waiting, support scheduling errors, or inefficient AGV assignments. |
| medium_agv_heavy_6v2 | Medium layout; 6 AGVs, 2 Pickers | Stronger AGV-heavy stress case combining larger layout, longer paths, support bottlenecks, and charging pressure. |
| small_picker_heavy_2v4 | Small layout; 2 AGVs, 4 Pickers | Tests whether surplus Picker capacity causes underutilisation, unnecessary motion, or role-confused support decisions. |

Table 3: Warehouse scenarios used in the objective experiments.

The scenario set was selected to vary two sources of difficulty. The first is spatial and temporal scale: the tiny, small, and medium layouts increase path length, number of shelf positions, and the number of simultaneous robot decisions. The second is role balance: balanced scenarios test ordinary collaboration, AGV-heavy scenarios make Picker support a scarce resource, and the Picker-heavy scenario tests whether extra support robots are used productively or create unnecessary motion. The scenario set supports the research questions by separating minimal coordination cases from settings with more agents, longer paths, support allocation, charging decisions, and conflict avoidance.

The language-model set was selected to provide a controlled cross-section of locally deployable, open-weight language models for instruction-following tasks. It represents

a selected benchmark subset of available LLMs. The evaluated models are llama3.2:1b, llama3.2:3b, mistral:7b, gemma3:12b, phi4, and qwen2.5:14b. These models span a range of parameter scales from approximately 1B to 14B parameters and originate from distinct model families developed by Meta, Mistral AI, Google, Microsoft, and Alibaba. Nominal parameter size was the primary selection criterion. Suitable model variants from a single family were not available across the full evaluated range, so the selected set spans both parameter scales and model families while keeping the simulator, prompt construction, action interface, and evaluation metrics fixed. This selection enables evaluation of whether performance increases consistently across the chosen off-the-shelf models as nominal parameter scale increases, or whether smaller locally deployable models provide comparable behaviour under the same experimental conditions. Accordingly, the results describe performance differences among the selected model configurations; parameter count is not treated as the sole cause of those differences.

Local execution was used for all selected models. All reported experiments were run on a fixed compute platform equipped with an NVIDIA RTX A4000 GPU. Keeping the hardware platform constant limits hardware-related variation between experimental configurations and makes the execution conditions explicit for reproducibility. Each experiment submits the generated prompt to a locally hosted Ollama instance, and the same set of models is used for the shared-context and centralized planners. Local inference reduces dependence on external cloud APIs, removes network availability from the experiment setup, and supports warehouse-robotics studies with deployment and connectivity constraints. These conditions define the execution boundary for this study. The objective is to compare planning behaviour across representative locally deployable language models.

The selected models are used in their released configurations, with no additional fine-tuning or parameter modification. Each prompt provides a task objective, structured state information, candidate actions, conflict warnings, and an output contract that is parsed back into simulator actions. The selected models are instruction-tuned or otherwise post-trained for instruction-following tasks [46, 47, 48, 49, 50]. This matches the constrained decision-making prompt used in this study. No model-specific prompt modifications are introduced. Apart from the selected model identifier, every experiment uses the same prompt templates, simulator state representation, parsing logic, validation layer, and evaluation metrics for a fixed objective, planning architecture, scenario, and prompt format. The reported experiment runners set the generation temperature to 0.1 and the maximum generated-token budget, num_predict, to 700. Top-p, top-k, and stop sequences are not overridden by the runners and therefore follow the locally installed Ollama model artefact. No separate language-model sampling seed is configured; the recorded seed controls simulator initialisation. Context length is obtained from the local Ollama model metadata when available. The effective context window

in this study is the value reported by the locally installed Ollama model artefact. Context use was measured using first-step prompts from medium_balanced_4v4, which contains eight agents and preserves the balanced AGV-Picker role ratio. The measured prompts cover centralized JSON, centralized natural-language, shared-context natural-language, and shared-context JSON prompting. Table 4 reports the locally configured context window, the prompt tokens used according to Ollama's prompt_eval_count, and the maximum percentage of the available context window used by these prompts.

| **Model** | **Local context window** | **Central JSON** | **Central NL** | **Shared NL** | **Shared JSON** | **Max use** |
|---|---|---|---|---|---|---|
| llama3.2:1b | 131072 | 8401 | 8201 | 2826 | 4763 | 6.4% |
| llama3.2:3b | 131072 | 8401 | 8201 | 2826 | 4763 | 6.4% |
| mistral:7b | 32768 | 11109 | 10894 | 3704 | 6357 | 33.9% |
| gemma3:12b | 131072 | 10252 | 10033 | 3354 | 5855 | 7.8% |
| phi4 | 16384 | 8386 | 8186 | 2811 | 4748 | 51.2% |
| qwen2.5:14b | 32768 | 9011 | 8804 | 3012 | 4958 | 27.5% |

Table 4: Locally reported Ollama context windows and prompt-token usage for representative prompts from medium_balanced_4v4.

The measured prompts fit within the locally reported context windows for all evaluated models. Each run produces a structured summary row containing both experimental identifiers and measured outcomes. Repeated executions with the same objective, architecture, prompt configuration, model, and scenario were first averaged into one configuration-level observation. Comparative analyses then used only observations with an available counterpart matching the remaining experimental factors, so configurations with additional repetitions did not receive greater weight. The metrics are reported directly or converted into derived metrics when a direct comparison would otherwise be misleading. Table 5 summarises how the reported metrics are used in the analysis.

| Metric | What it measures | Analytical role |
|---|---|---|
| Mean shelf deliveries | Measures how much of the common shelf-delivery task is completed before full completion or sustained inactivity. | Primary outcome for the main RQ and RQ1–RQ3. |
| Mean LLM calls | Measures the number of model requests made during a run. | Secondary planning-overhead measure, especially for RQ2 and RQ3. |
| Deliveries per 1000 LLM calls | Measures completed deliveries relative to the frequency of model interaction. | Secondary call-normalised measure for RQ1–RQ3. |
| Battery-aware productivity | Describes delivery performance together with charging-station utilisation. | Exploratory objective-sensitivity analysis; not a primary RQ outcome. |
| Minimum battery, below-threshold fraction, and zero-battery fractions | Describe the lowest observed battery level, the fraction of agent-steps below 30, and the fractions of agents and runs reaching zero. | Direct battery-outcome comparison; not a primary RQ outcome. |

**Table 5** Mapping between the reported metrics and their roles in the analysis.

Invalid-action and generation-failure fields are not treated as research-question outcomes; they are retained for diagnostic run validation as described in the delimitations.

Deliveries per 1000 LLM calls is computed as:

$$\text{Deliveries per 1000 calls} = \frac{\text{Mean deliveries}}{\text{Mean LLM calls}} \times 1000 \tag{1}$$

This metric is used mainly for prompt-format comparisons, where natural-language and JSON prompts can produce different numbers of calls and different run lengths. It normalises delivery output by the frequency of model interaction; it is not a measure of token use, latency, energy consumption, or computational cost. For centralized versus shared-context comparisons, raw mean deliveries are treated as the primary performance measure because shared-context planning structurally makes more LLM calls by querying per agent. In that comparison, call-based metrics describe model-interaction frequency and are not treated as a common computational-cost unit.

For exploratory battery analysis, a battery-aware productivity score is used:

For each run, charging-station utilisation is the sum of occupied charging stations across recorded steps divided by the sum of available charging stations across those steps. The

resulting fraction is reported as a percentage. The score uses this percentage value:

$$\text{Battery-aware productivity} = \text{Mean deliveries} \times \text{Mean charging-station utilisation (\%)} \tag{2}$$

This is a descriptive index expressed in delivery–percentage-point units, not a percentage, and it is therefore not bounded by 100. It indicates whether charging-station occupancy occurred together with delivery performance, but it does not distinguish necessary charging from unnecessary charging and is not a measure of battery-management optimality.

The direct battery comparison also reports the mean minimum battery observed, the fraction of agent-steps with battery below 30, the fraction of agents reaching zero, and the fraction of runs in which at least one agent reached zero.

## 3.6 Experimental Protocol

In this thesis, a *run* is one simulator execution under a specified objective, architecture, prompt configuration, model, and scenario. A *session* is a batch invocation that may execute several such runs. A completed run is retained when it follows the reported protocol; runs affected by technical or configuration failures are excluded as described in the delimitations.

The experiment sessions were run with fixed environment IDs and logged run configurations, creating a controlled set of comparable runs for the reported objective families. The simulator seed is recorded for reproducibility and traceability. The execution configuration uses `--max_steps 0`, which disables the fixed maximum-step cap. Runs terminate through inactivity control, with `--max_inactivity_steps 500`. This setting allows a run to continue while agents keep making progress until the inactivity condition is reached. In this simulation setup, a run can achieve at most 20 deliveries, and the results chapter uses that ceiling to contextualise reported scores.

**Run procedure.** Each reported run follows the same overall procedure:

1. Select the objective, scenario, model, planning architecture, prompt configuration, and recorded simulator seed.
2. Initialise Battery-TA-RWARE and the locally hosted Ollama model using the corresponding runner configuration.

3. Execute the state–candidate–prompt–query–validate–action loop defined earlier in this chapter.

4. Continue until all 20 requested shelves are delivered or 500 consecutive steps pass without a delivery.

5. Store the step records and run summary, including configuration identifiers, deliveries, executed steps, LLM calls, invalid or missing model actions, generation failures, and elapsed time.

6. Average repeated executions as described above, then use matched configurations for each comparison.

The main standardised controller settings are summarised in Table 6. These settings define the stopping condition, the degree of replanning allowed during long-running assignments, and the path-awareness assumptions used when constructing candidate actions.

| **Setting** | **Value** | **Methodological role** |
|---|---|---|
| Episode step cap | Disabled | Uses no fixed maximum-step horizon. A run can continue while deliveries remain possible. |
| No-delivery termination | 500 steps | Terminates an episode after 500 consecutive steps with no shelf delivery. This limits stalled runs while allowing difficult scenarios time to recover from temporary congestion. |
| Busy-agent replanning threshold | 10 steps | Allows an eligible busy agent to be reconsidered after 10 consecutive busy steps. This supports recovery from stale or blocked macro assignments while keeping selective replanning. |
| Proactive support signal | Disabled | Limits Picker-support prompting to cases where an AGV is currently waiting for support. This keeps the prompt focused on immediate coordination needs. |
| Agent-aware path finding | Enabled | Computes paths while considering other agents as obstacles. This makes candidate distances and reachability estimates more conservative in congested states. |

**Table 6** Standardised controller settings used in the reported experiments.

The `--prompt_format` argument selects either the natural-language or JSON prompt. The selected planning architecture determines whether the controller uses centralized

or shared-context decision-making. The `--selected_models` argument selects the model or models for a session, while the results and session directories store the aggregated outputs and detailed logs.

The protocol defines the analysis structure used in the thesis. When comparing centralized and shared-context planning, the same scenarios and model families are used, and call metrics are interpreted with the per-agent query structure of shared-context planning in mind.

## 3.7 Delimitations

To keep the study focused and interpretable, the following delimitations apply:

- The work is evaluated in a simulated warehouse environment.
- The planner receives symbolic simulator state without sensor noise or perception errors. The study therefore does not establish performance on perception-limited physical robots.
- All language-model inference uses Ollama on one NVIDIA RTX A4000 system. Portability across inference servers and hardware platforms was not evaluated.
- The findings apply to the modified Battery-TA-RWARE environment used in this study and are not treated as direct benchmark comparisons with published TA-RWARE results or other simulator families.
- The controller is responsible only for high-level target selection; low-level navigation and collision handling are performed by the environment.
- The evaluated language models are prompted general-purpose models accessed through a local inference server, with no task-specific fine-tuning applied.
- The reported experiments use a limited number of completed sessions and simulator initialisations, so the analysis is descriptive.
- Only runs using the protocol described in this chapter are included; outputs from configurations with different stopping or control settings are excluded from the objective-comparison analysis.
- Invalid-action and generation-failure fields are used for diagnostic run validation. Logging these fields helped identify technical and configuration errors, allowing inconsistent runs to be excluded from the analysis.

# 4 Results and Discussion

## 4.1 Experimental Data Overview

The results are organised around the three research questions introduced in Section 1.1. The analysis focuses on three main comparison axes: prompt configuration, planning architecture, and model behaviour across scenario configurations. In addition, an exploratory analysis is included for objective sensitivity.

The metrics and experimental conditions are defined in Section 3.5. This chapter reports raw shelf deliveries and LLM calls together with the call-normalised delivery measure defined in Eq. 1. Raw delivery count remains the main task-completion measure, while the call-normalised value describes deliveries relative to model-interaction frequency. The analyses use matched configuration-level observations drawn from the Calls, Battery + Shelf, and Battery objective families. Repeated executions with the same objective, architecture, prompt configuration, model, and scenario identifiers are averaged before matching, and therefore do not receive additional weight.

## 4.2 RQ3: Prompt-Configuration Comparison

RQ3 compares collaborative task performance under the tested natural-language and JSON-based prompt configurations. Table 7 reports the scenario-level comparison, using 144 matched configuration-level observations for each prompt configuration. Across all six scenarios, the natural-language configuration produced higher mean deliveries and higher deliveries per 1000 calls than the JSON-based configuration.

| Scenario | JSON mean del. | JSON del./1000 calls | Natural mean del. | Natural del./1000 calls |
|---|---|---|---|---|
| tiny_balanced_1v1 | 1.96 | 3.54 | 4.33 | 7.45 |
| small_balanced_2v2 | 0.65 | 1.33 | 3.40 | 4.23 |
| small_agv_heavy_4v2 | 6.44 | 4.85 | 11.08 | 7.41 |
| small_picker_heavy_2v4 | 4.90 | 2.85 | 6.60 | 3.88 |
| medium_balanced_4v4 | 4.42 | 2.99 | 8.21 | 4.72 |
| medium_agv_heavy_6v2 | 3.83 | 2.90 | 9.29 | 7.15 |

**Table 7** Prompt-configuration comparison by scenario. Each cell summarises 24 configuration-level observations per prompt, matched by objective, planning architecture, model, and scenario.

The same overall pattern appears when grouping by model, as shown in Table 8. The natural-language configuration produced higher mean deliveries and call-normalised delivery values for five of the six models. Phi4 was the exception on both measures, with

5.81 mean deliveries and 8.03 deliveries per 1000 calls under JSON, compared with 5.79 and 7.70 under natural language. The differences were especially pronounced for the two Llama models and Qwen.

| Model | JSON mean del. | JSON del./1000 calls | Natural mean del. | Natural del./1000 calls |
|---|---|---|---|---|
| llama 1b (3.2) | 0.75 | 0.76 | 2.81 | 2.72 |
| llama 3b (3.2) | 2.33 | 1.90 | 12.08 | 6.17 |
| mistral 7b | 4.42 | 3.76 | 7.54 | 4.36 |
| gemma 12b | 4.21 | 2.60 | 7.69 | 7.16 |
| phi4 | 5.81 | 8.03 | 5.79 | 7.70 |
| qwen 14b (2.5) | 4.67 | 4.09 | 7.00 | 6.52 |

**Table 8** Prompt-configuration comparison by model. Each model and prompt configuration uses $n = 24$ configurations matched by objective, planning architecture, and scenario.

These results suggest that the evaluated models were better able to use the natural-language state description than the JSON representation in this implementation. One possible explanation is that converting the simulator state into descriptive sentences made relationships among agents, targets, and constraints more explicit. Although the same state variables were available in the JSON prompt, their meaning had to be inferred from field names, values, and the structure of the object. The natural-language representation may therefore have provided additional linguistic context for models whose pretraining is primarily based on large text corpora.

The result is also consistent with Tam *et al.* [36], who found that restricting model outputs to structured formats, including JSON, could reduce performance on reasoning tasks. Their study also showed that the effect was task-dependent and that structured formats could remain competitive on classification tasks. In the present experiments, the JSON condition changed both the state representation and the output constraint, so the observed difference cannot be attributed to either factor in isolation. A model trained or post-trained with greater exposure to structured data might behave differently. Moreover, JSON can encode relationships through meaningful field names and hierarchy, but the results indicate that this encoding was less effective than the descriptive representation for the models and planning interface evaluated here.

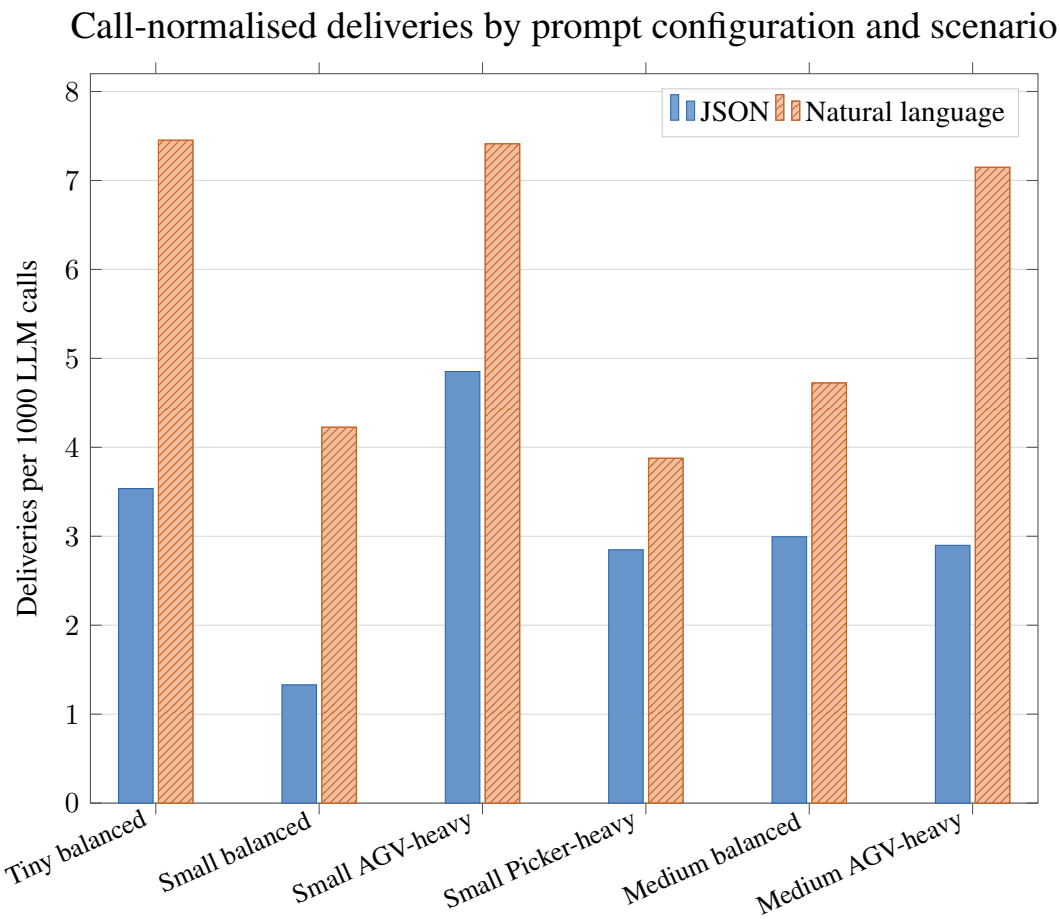


**Figure 7** Call-normalised deliveries by prompt format across scenarios. Each bar is calculated from 24 matched configuration-level observations. Higher bars indicate more deliveries per 1000 model calls.

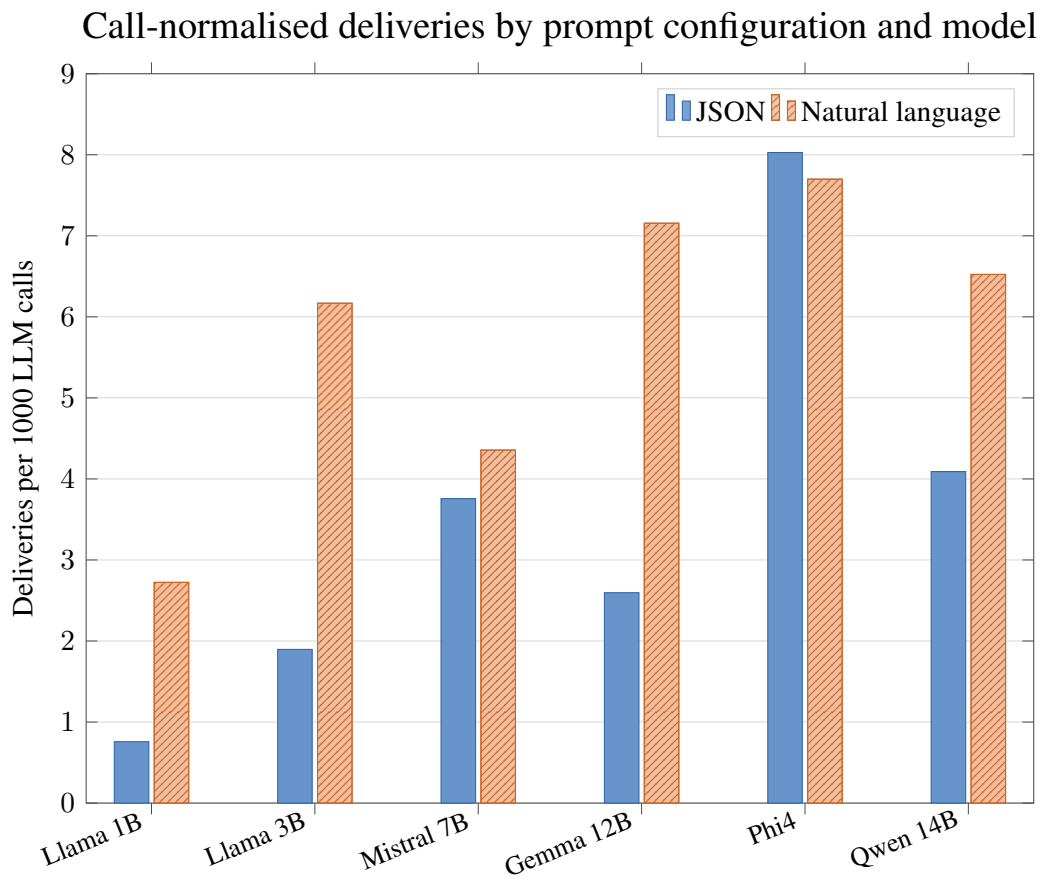


**Figure 8** Call-normalised deliveries by prompt format across models. Each model and prompt configuration uses $n = 24$ matched configuration-level observations. Higher bars indicate more deliveries per 1000 model calls.

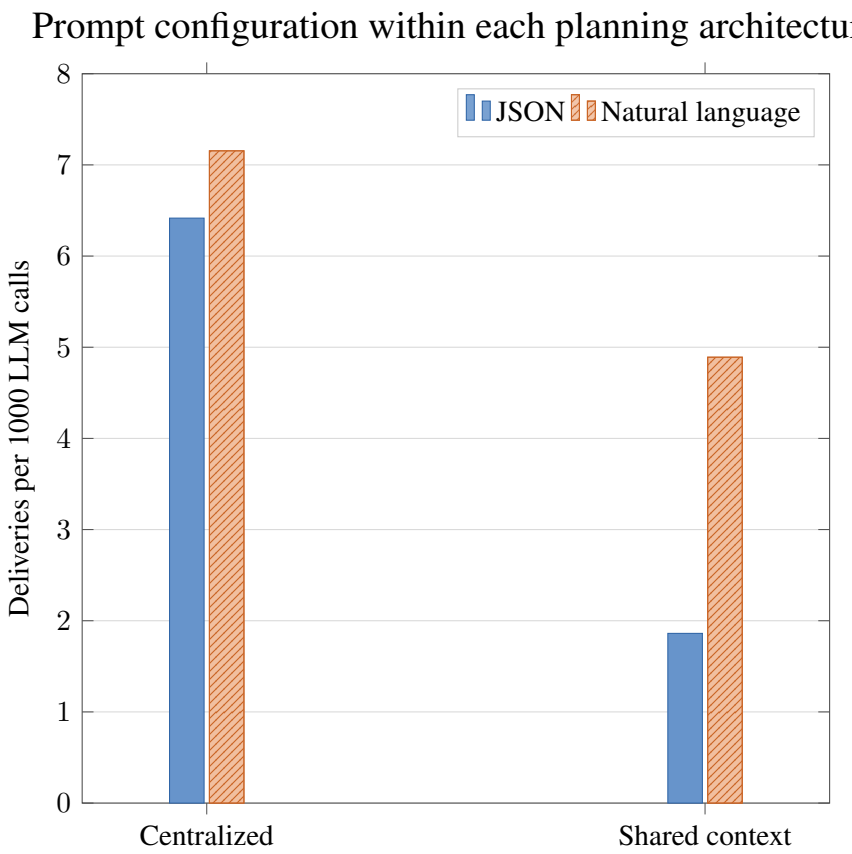


**Figure 9** Call-normalised deliveries by prompt format within each planning architecture. Each bar is calculated from 72 matched configuration-level observations. Higher bars indicate more deliveries per 1000 model calls; the measure describes interaction frequency rather than computational cost.

These results show that the tested natural-language prompt configuration, which presented the simulator state in descriptive form, supported better collaborative task performance overall than the tested JSON-based configuration. It produced higher mean deliveries and higher deliveries per 1000 model calls across all six scenario groups and for five of the six models. Phi4 showed a small JSON advantage on both measures. The consistency of the broader pattern supports the answer to RQ3: at the level of the complete prompt configurations, the descriptive natural-language configuration was more effective overall, although the Phi4 result shows that the effect was model-dependent and cannot be attributed to a single formatting component.

## 4.3 RQ2: Planning-Architecture Comparison

RQ2 compares centralized planning with shared-context per-agent planning. Because shared-context planning can call the LLM separately for multiple agents, LLM calls are partly a consequence of the architecture itself. For this reason, the architecture comparison is interpreted primarily through mean deliveries, while the call-normalised value is treated as a secondary measure of model-interaction frequency rather than computational cost.

Table 9 reports the scenario-level comparison, with 16 matched configuration-level observations per architecture within each scenario. Shared-context planning produced more mean deliveries in every scenario, although it also made more LLM calls. Centralized planning therefore used fewer model interactions, while shared-context planning produced higher delivery counts in this dataset.

| Scenario | Central mean calls | Central mean del. | Shared mean calls | Shared mean del. |
|---|---|---|---|---|
| tiny_balanced_1v1 | 605.31 | 3.08 | 717.89 | 6.05 |
| small_balanced_2v2 | 598.54 | 2.67 | 1230.46 | 4.54 |
| small_agv_heavy_4v2 | 925.89 | 8.67 | 2185.60 | 13.15 |
| small_picker_heavy_2v4 | 939.61 | 3.86 | 2551.91 | 8.93 |
| medium_balanced_4v4 | 1297.57 | 7.83 | 2412.21 | 8.18 |
| medium_agv_heavy_6v2 | 816.61 | 7.69 | 1852.57 | 10.59 |

**Table 9** Planning architecture comparison by scenario. Each architecture–scenario cell summarises 16 configuration-level observations matched by objective, prompt configuration, model, and scenario.

The model-level architecture comparison uses 12 matched configuration-level observations per architecture for each model. Shared-context planning produced more mean deliveries for the Llama models, Mistral, Gemma, and Phi4, while Qwen produced more under centralized planning. The delivery gap between the two architectures generally narrowed among the larger selected models, with Qwen reversing the overall pattern. This indicates that the best planning architecture may depend on both model behaviour and scenario size.

One possible explanation for the overall shared-context advantage is that each model call selects an action for only one agent. This may reduce the number of simultaneous decisions that must be produced and avoids requiring the model to construct a mutually compatible joint action in a single response. Centralized planning nevertheless receives the complete shared state and, in principle, can account for dependencies among all agents directly. The results show that access to global information alone did not produce higher delivery performance under the centralized prompt used here; the model also had to integrate that information successfully into a joint plan.

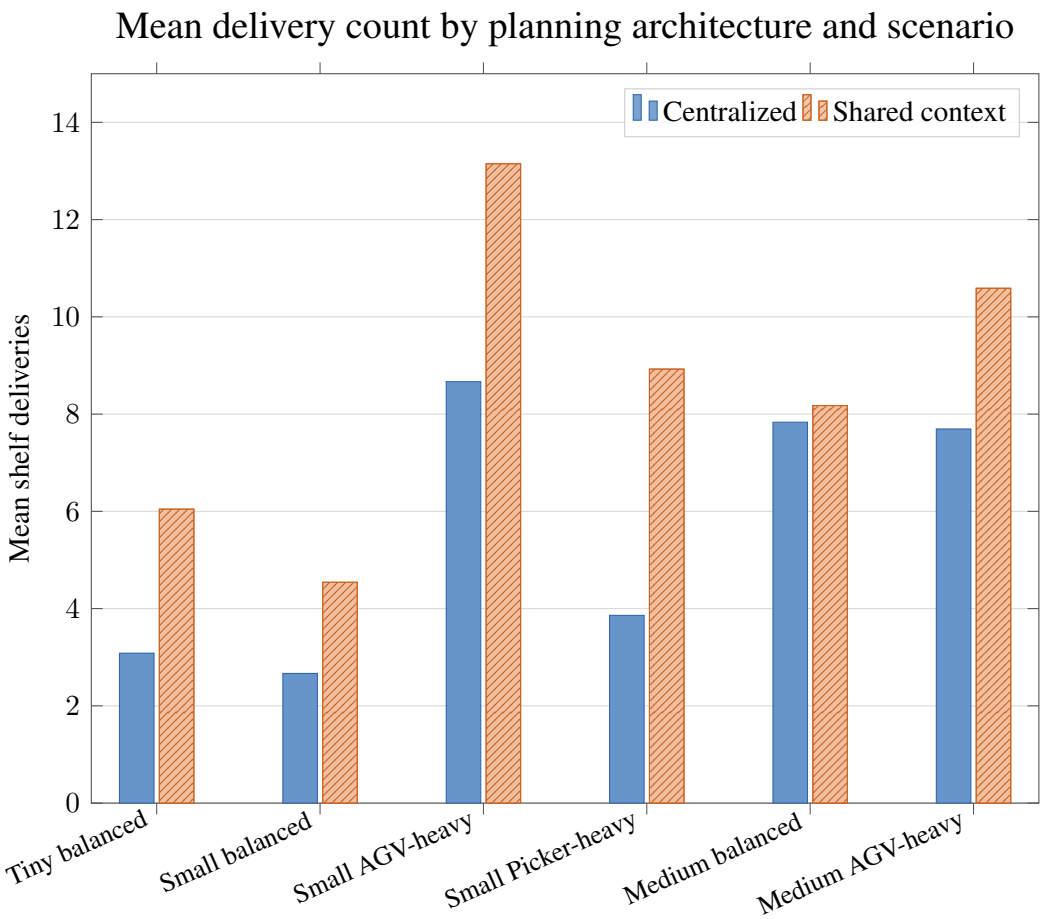


**Figure 10** Centralized and shared-context mean deliveries by scenario. Each bar is the mean of 16 matched configuration-level observations per architecture and scenario.

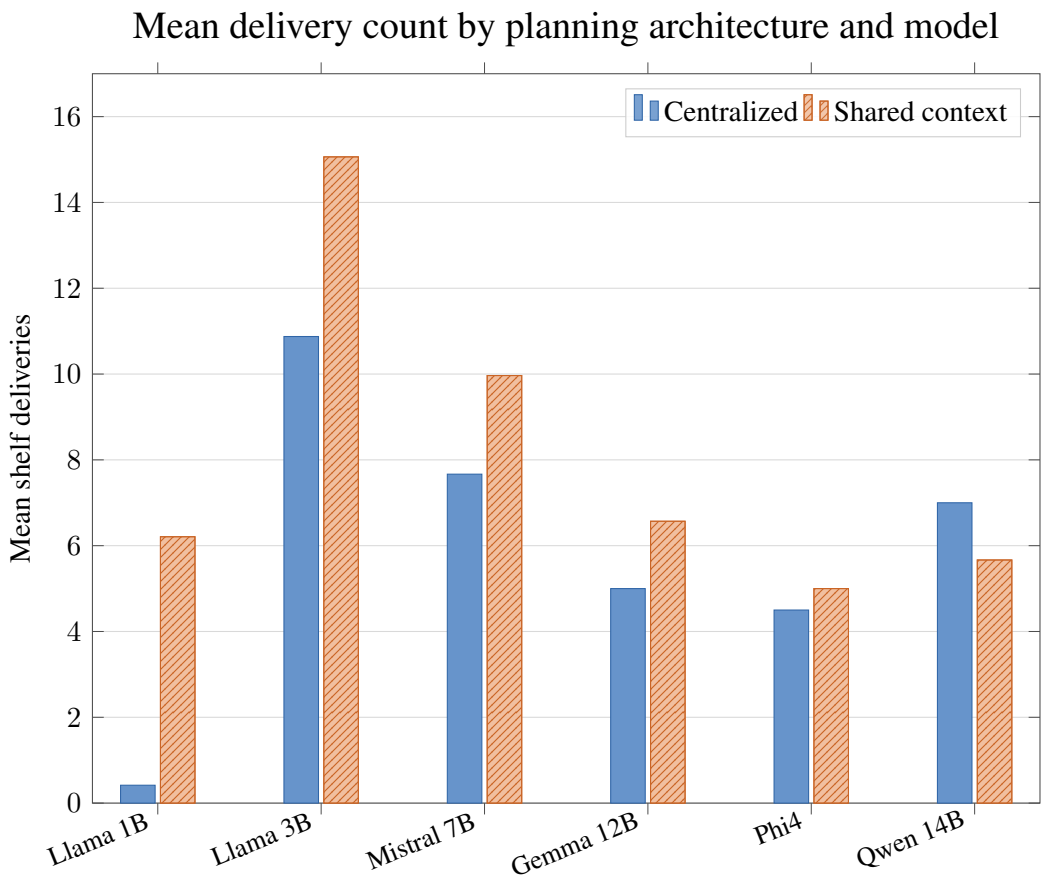


**Figure 11** Centralized and shared-context mean deliveries by model. Each bar is the mean of 12 matched configuration-level observations per architecture and model.

Across the selected model artefacts ordered by nominal parameter size, the delivery difference between shared-context and centralized planning generally narrowed, and Qwen reversed the pattern by producing more deliveries under centralized planning. This trend is compatible with the hypothesis that some models handle the additional joint-planning demand more effectively. Related work has reported stronger performance from a larger backbone in planning-based logical reasoning [51], while evaluations of autonomous language-model planning have also found substantial differences between models and identified the strongest tested model as the best performer, although its absolute success rate remained limited [52]. These findings support treating planning capability as model-dependent, but they do not establish parameter count as the cause of the present architecture differences because the evaluated models also differ in model family and post-training. Across the full architecture comparison, shared-context planning produced higher mean deliveries in all six scenario groups and for five of the six selected

models. The consistency across different layouts, agent populations, and role balances indicates that this advantage was not restricted to one type of collaboration demand. RQ2 therefore reveals a trade-off: shared-context planning generally produced higher delivery performance in the tested setting, whereas centralized planning required fewer model interactions; the narrowing model-level gap and Qwen reversal show that the preferred architecture remains model-dependent.

## 4.4 RQ1: Model Choice and Scenario Complexity

RQ1 asks how grounded collaborative task performance varies among the selected locally deployable models spanning different nominal parameter sizes. The model-level evidence is considered across the prompt-configuration comparison in Table 8 and Figure 8, the architecture comparison in Figure 11, and the broader matched comparison in Figure 12.

For the broader comparison, a configuration is included only when all six models have observations for the same objective, planning architecture, prompt configuration, and scenario. This gives 48 configuration-level observations per model, divided equally between 24 natural-language and 24 JSON configurations. The same configurations and prompt-format balance are therefore represented for every model in Figure 12.


**Figure 12** Delivery performance across the matched model comparison. Each model uses the same 48 objective–architecture–prompt–scenario configurations, comprising 24 natural-language and 24 JSON configurations. The second measure reports deliveries per 1000 model calls.

Figure 12 shows that llama 3b achieved the highest mean delivery count, at 7.21, while Phi4 achieved the highest call-normalised value, at 7.86 deliveries per 1000 calls. Llama 1b produced the lowest values on both measures. Mistral, Gemma, and Qwen achieved similar mean delivery counts, but their call-normalised values differed.

The prompt-configuration comparison provides a second model-level view. Table 8 and Figure 8 show higher natural-language mean deliveries and call-normalised values for five models; Phi4 instead produced slightly higher values under JSON on both measures. The size of the prompt-related difference also varied substantially by model. Llama 3b achieved the highest natural-language mean delivery count, while Phi4 achieved the highest natural-language call-normalised value in the updated matched subset. The architecture comparison in Figure 11 also shows a model-dependent pattern: shared-

context planning produced more deliveries for five models, whereas Qwen produced more under centralized planning. Model performance therefore changed with both prompt configuration and planning architecture.

Scenario complexity also affected performance. Natural-language prompts produced the highest call-normalised delivery values in the tiny balanced and small AGV-heavy scenarios, with a similarly high value in the medium AGV-heavy scenario, while the values were lower in the small picker-heavy and small balanced scenarios. This suggests that model performance is shaped by the interaction between model behaviour, prompt representation, planning architecture, and the collaboration demands of the scenario.

The comparatively low performance of some larger models should not be interpreted as evidence that larger models are intrinsically less suitable for collaborative planning. Rather, the results show that, for some of the evaluated configurations, smaller models achieved task performance comparable to or better than that of larger models. Since smaller models generally require less memory and inference computation, they may provide a favourable performance–resource trade-off for locally deployed planners [44]. Computational cost was not measured directly in this comparison, so the present result establishes competitive task performance rather than a quantified cost reduction. It nevertheless indicates that increasing nominal parameter count is not always necessary to obtain effective performance in a fixed collaborative planning setting.

Overall, the results answer RQ1 at the level of the selected model artefacts: model choice mattered, but performance did not increase consistently with nominal parameter size. Llama 3b produced the highest mean delivery count in the balanced 48-configuration comparison, while Phi4 produced the highest call-normalised value. Larger models such as Gemma 12b and Qwen 14b did not uniformly dominate these measures. Since all six models were compared over the same 24 natural-language and 24 JSON configurations, these changing rankings support the conclusion that nominal parameter size alone did not predict collaborative task performance among the selected model artefacts.

## 4.5 Model Adaptability to Objectives

This analysis compares three objective families: reducing LLM calls, balancing shelf delivery with battery awareness, and prioritising battery management to examine how model behaviour changed with the stated objective.

Table 10 summarises the aggregate objective-level results for 72 matched configurations per objective. Battery produced the lowest mean number of calls, while Battery + Shelf produced the highest mean deliveries and call-normalised delivery value. Calls

produced slightly fewer calls than Battery + Shelf, but also fewer deliveries and a lower call-normalised value.

| Objective | Matched combinations | Mean calls | Mean deliveries | Deliveries/1000 calls |
|---|---|---|---|---|
| Calls | 72 | 1267.54 | 6.82 | 5.38 |
| Battery + Shelf | 72 | 1272.45 | 7.48 | 5.88 |
| Battery | 72 | 1241.86 | 6.51 | 5.24 |

**Table 10** Objective-level calls and deliveries across configurations matched among the Calls, Battery + Shelf, and Battery objectives.

To examine whether models adapted to the call-reduction objective, the analysis compares each model under the matched Calls and Battery + Shelf configurations. The mean-call difference in Figure 13 is computed as Calls minus Battery + Shelf, so negative values indicate fewer calls under the Calls objective. Llama 1b, llama 3b, Phi4, and Qwen produced fewer calls under Calls, whereas Mistral and Gemma produced more. The response to the call-reduction objective was therefore model-dependent rather than consistent across all models.

Battery-related behaviour was examined using the descriptive battery-aware productivity score defined in Eq. 2. The score indicates whether charging-station occupancy occurred together with successful delivery performance; it does not distinguish necessary from unnecessary charging and is not an optimality measure. Table 11 reports this score across the matched three-objective configurations. The Battery objective produced the highest score for llama 1b and Mistral, Battery + Shelf produced the highest score for llama 3b, Phi4, and Qwen, and Calls produced the highest score for Gemma. The differing rankings show that the Battery objective did not produce a consistent improvement across the evaluated models.

| Model | Calls score | Battery + Shelf score | Battery score |
|---|---|---|---|
| llama 1b (3.2) | 41.44 | 48.76 | 51.54 |
| llama 3b (3.2) | 135.36 | 214.35 | 207.94 |
| mistral 7b | 98.65 | 153.43 | 166.25 |
| gemma 12b | 106.92 | 84.98 | 58.05 |
| phi4 | 50.14 | 76.99 | 15.83 |
| qwen 14b (2.5) | 95.19 | 104.79 | 100.24 |

**Table 11** Battery-aware productivity by model and objective, using the descriptive score defined in Eq. 2. Each model–objective cell uses $n = 12$ matched configuration-level observations.

A direct battery-outcome comparison was made across 72 matched natural-language

configurations per objective. The Battery objective produced a higher mean minimum battery and slightly lower zero-battery fractions. Its below-threshold fraction was slightly higher, while mean deliveries and charging-station utilisation were lower.

For each configuration, the minimum battery is the lowest value reached by any agent during the run. The agent-step fraction is the proportion of all agent-step observations below the critical threshold of 30. The agent-zero fraction is the proportion of agents that reached zero battery, and the run-zero fraction is the proportion of configurations in which at least one agent reached zero. The table reports the mean of each measure across the matched configurations.

| Objective | Matched | Mean del. | Charger util. (%) | Mean min. battery | Agent-step frac. $< 30$ | Agent-zero frac. | Run-zero frac. |
|---|---|---|---|---|---|---|---|
| Battery | 72 | 6.66 | 13.91 | 5.01 | 0.292 | 0.442 | 0.736 |
| Battery + Shelf | 72 | 7.26 | 14.87 | 3.70 | 0.286 | 0.475 | 0.750 |
| Battery − Battery + Shelf | — | $-0.60$ | $-0.95$ | $+1.31$ | $+0.006$ | $-0.033$ | $-0.014$ |

**Table 12** Direct battery outcomes across matched Battery and Battery + Shelf configurations. Repeated runs are averaged before each configuration receives equal weight.

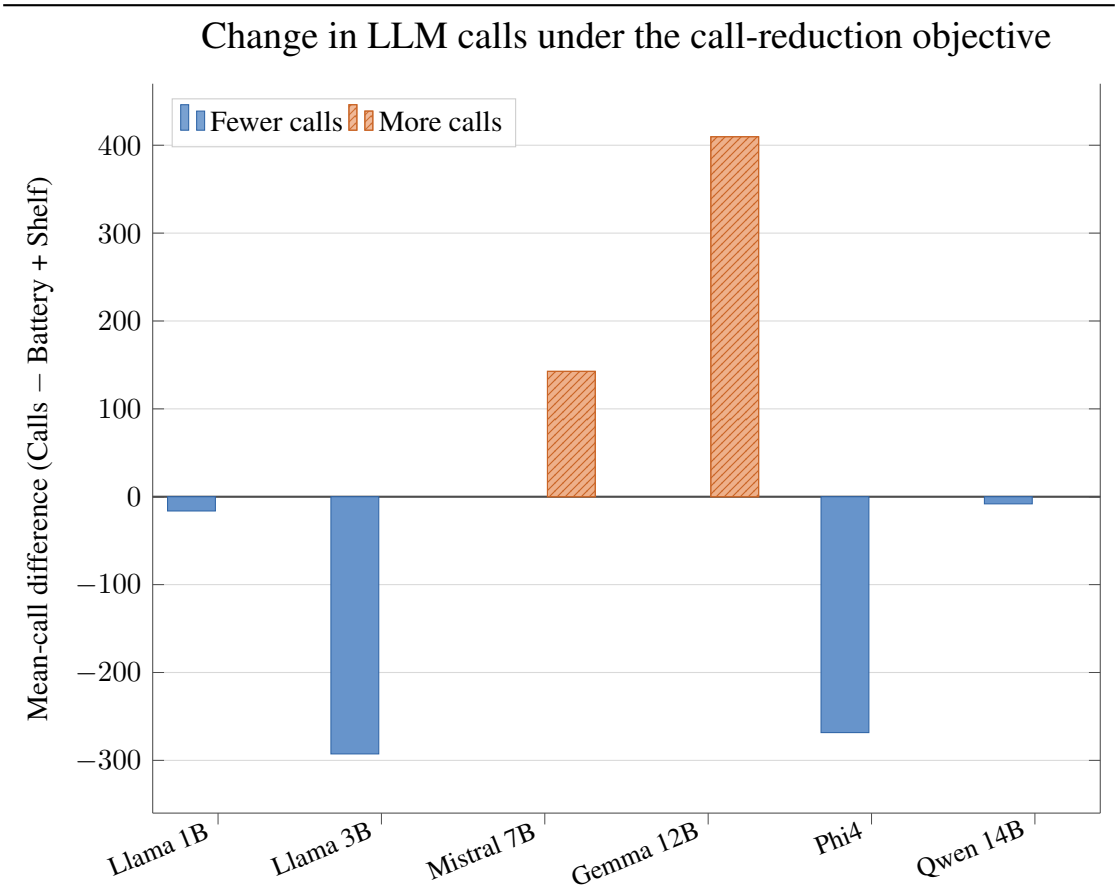


**Figure 13** Model response to the LLM call-reduction objective. Mean-call difference is computed as Calls minus Battery + Shelf; negative values indicate fewer calls under Calls. Each model–objective mean uses $n = 24$ matched configuration-level observations.

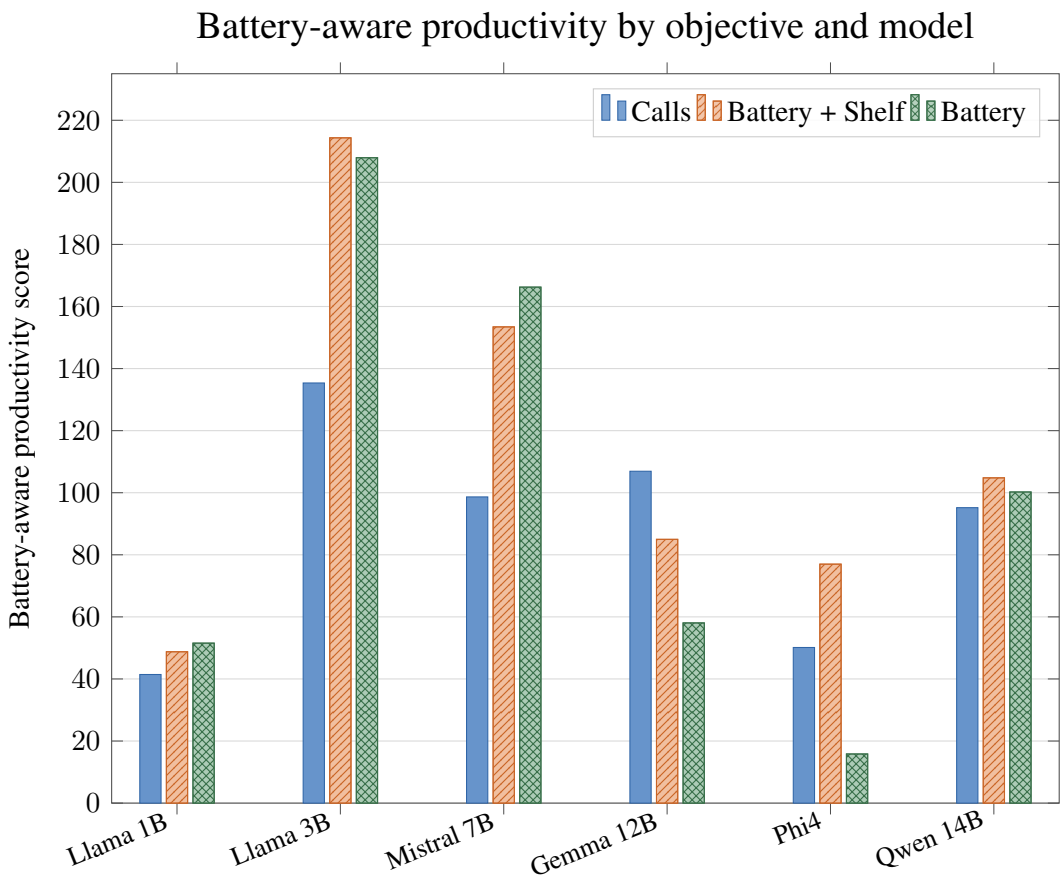


**Figure 14** Battery-aware productivity across objective prompts. Higher bars indicate that charging utilisation occurred together with more deliveries; the score is descriptive rather than an optimality measure. Each bar uses $n = 12$ matched configuration-level observations.

The objective changes produced mixed, model-dependent responses. Under the Calls objective, four models reduced their mean number of calls, while Mistral and Gemma increased it. Across the matched pairwise comparison, the aggregate reduction relative to Battery + Shelf was small, and Battery produced the lowest mean call count in the three-objective comparison. The Calls instruction therefore affected the models differently but did not consistently achieve its stated purpose. For the Battery objective, the mean minimum battery increased by 1.31 and the agent-zero and run-zero fractions decreased slightly, but the below-threshold fraction increased by 0.006 and mean deliveries decreased by 0.60. These opposing results suggest a small change in battery-related behaviour rather than a general improvement in battery management. Since the model weights and controller were unchanged between the matched objective configurations, the observed differences support sensitivity to the stated objective, but not consistent

adaptation to the requested objectives.

## 4.6 Summary of Findings

The main research question is answered positively within the tested simulator: prompted LLMs can support grounded high-level decision-making for heterogeneous robot collaboration. Successful shelf deliveries were observed across the evaluated models, architectures, prompt configurations, and scenarios. This finding supports the answer because it shows that the generated decisions could be validated, executed, and used to complete collaborative tasks. It establishes feasibility within Battery-TA-RWARE, but not superiority over non-LM planning methods.

For RQ1, the selected model affected collaborative task performance, but nominal parameter size alone did not predict performance. Llama 3b achieved the highest mean delivery count and Phi4 the highest call-normalised value in the matched 48-configuration comparison, while the larger Gemma and Qwen models did not consistently dominate. This finding supports the answer because all models were evaluated over the same balanced set of 24 natural-language and 24 JSON configurations, yet their performance did not increase with parameter size.

For RQ2, shared-context planning generally produced higher delivery performance, while centralized planning required fewer model interactions. Shared-context planning produced higher mean deliveries in all six scenario groups and for five of the six models. Across the selected model artefacts ordered by nominal parameter size, the delivery difference between the architectures generally narrowed, with Qwen eventually producing more deliveries under centralized planning. This finding supports the answer because the shared-context advantage appeared across every scenario group, while the narrowing model-level difference and Qwen reversal show that the architecture effect remains model-dependent. Since the models belong to different families, the trend does not establish parameter size as its cause.

For RQ3, the complete natural-language prompt configuration was more effective overall than the complete JSON-based configuration. Natural language produced higher mean deliveries and call-normalised values in all six scenario groups and for five of the six models; Phi4 showed a small JSON advantage on both measures. This finding supports the answer because the same overall pattern appears in the scenario-level and most model-level matched comparisons. The result applies to the complete prompt configurations and does not isolate the input representation from the output constraints.

The supplementary objective-sensitivity analysis did not show consistent adaptation. The call-reduction objective did not consistently reduce LLM usage, and the direct bat-

tery indicators showed only small, mixed differences between the Battery and Battery + Shelf objectives. These findings support only limited objective sensitivity and do not provide conclusive evidence of adaptation.

# 5 Ethics and Sustainability

Although the present work is simulation-based, it raises practical issues that remain relevant for real multi-robot deployments. The primary ethical concern is safety and reliability: generated decisions may be infeasible, unstable, or unproductive if they are not constrained by system-side checks. The studied control loop executes only grounded action identifiers after candidate shaping, feasibility validation, and fallback handling. Actions invalid under the current simulator mask are filtered before execution. This feasibility validation does not guarantee avoidance of unproductive behaviour such as deadlock, starvation, or battery depletion. The logging of prompts, parsed outputs, executed actions, and observed step consequences also improves traceability, which is important when analysing failures such as long no-delivery tails, stale waiting loops, or battery-related breakdowns.

From a sustainability perspective, the main trade-off is between model capability and computational cost. Larger language models generally require more inference time and compute, while smaller models may require stronger prompt structure and validation to remain effective. This thesis therefore treats call efficiency, unnecessary replanning, and coordination overhead as experimental outcomes with practical relevance. Even with safeguards, limited simulator initialisations, limited scenario coverage, and the absence of physical deployment still bound the scope of the reported results.

# 6 Conclusions and Future Work

This thesis evaluates LLM-driven decision-making for heterogeneous multi-robot collaboration in a task-assignment warehouse setting with battery constraints. The implemented control loop grounds decisions to discrete environment action identifiers, queries an LLM using centralized or per-agent shared-context prompts, and enforces feasibility through action-mask validation.

The reported comparisons use matched configuration-level observations drawn from the call-aware, battery-plus-shelf, and battery-aware objective families. Prompt-configuration, planning-architecture, and objective-sensitivity results include only configurations with the required counterpart, and repeated executions with the same configuration identifiers are averaged before comparison.

The experiments answer the main research question by showing that prompted LLMs can produce grounded high-level decisions and complete collaborative shelf deliveries within the tested simulator framework. The occurrence of successful deliveries across models, architectures, prompt configurations, and scenarios supports the conclusion that this approach is feasible within the tested setting, while the absence of a non-LM baseline prevents a claim of superiority over other planning methods. For RQ1, collaborative task performance across the selected model artefacts did not increase consistently with nominal parameter size in the balanced 24-natural and 24-JSON comparison. For RQ2, shared-context planning produced higher mean deliveries in all six scenario groups and for five of the six selected models, while centralized planning used fewer LLM calls. For RQ3, the tested natural-language prompt configuration produced higher mean deliveries and call-normalised delivery values across all six scenario groups and for five of the six models; Phi4 showed a small JSON advantage on both measures. The objective-sensitivity analysis showed mixed adaptation: the call-aware objective did not consistently reduce LLM usage, and the direct battery indicators showed small, mixed differences between the Battery and Battery + Shelf objectives.

## 6.1 Future Work

The capabilities of language models and the systems built around them are developing rapidly. Consequently, the results in this thesis represent one set of models and configurations at a particular point in time, rather than a final assessment of language-model-based robot collaboration. Model choice, decoding parameters, prompt design, context construction, and tool access can all affect the resulting behaviour. Future evaluations should therefore treat these choices as experimental variables and document them

sufficiently to distinguish improvements in the underlying model from improvements in the surrounding system.

One direction is to extend the planner with retrieval-augmented generation and external tools exposed through Model Context Protocol servers. Retrieval could provide task-relevant information without placing all available knowledge in every prompt, while reusable skills could define how the model accesses simulator state, robot capabilities, operational rules, or previous experience. Such additions should be evaluated separately from the base model because they alter the information and actions available to the planner. Of particular interest is whether they improve grounded decision-making and generalisation to new warehouse configurations, rather than only increasing system complexity.

The shared-context architecture studied here gives each agent access to a common state description, but it does not investigate communication as a behaviour in its own right. A further study could allow agents to communicate their local state, intended action, or request for assistance directly to one another. With a retained communication history or an explicit adaptation mechanism, repeated interactions could then be examined to determine whether the agents converge towards a useful communication protocol: what information is sent, when communication occurs, and which agents need to receive it. Comparing unrestricted communication with bandwidth-, latency-, or cost-constrained communication would help establish whether the exchanged messages contribute to collaboration or merely duplicate information already available in the environment state.

Deployment-oriented model optimisation is another open direction. Tools such as Embedl[2], which target the optimisation of models for specific hardware platforms, may reduce latency, memory use, or energy consumption and thereby make local inference more practical. However, techniques used to obtain these gains may also change model behaviour. It would therefore be useful to evaluate the unmodified and optimised versions of a model on the same scenarios, measuring not only execution performance but also action validity, planning quality, robustness, and collaborative task performance. This would reveal whether hardware-specific optimisation introduces a capability loss that is relevant to robot decision-making [44].

The experiments reported in this thesis were executed on an NVIDIA RTX A4000. Repeating the same protocol on other GPU generations, accelerator architectures, and resource-constrained edge hardware would test the portability of the findings. The comparison should keep the model artefact, inference configuration, and simulator conditions fixed where possible, while recording differences in numerical precision, runtime, memory use, and energy consumption. This would help determine whether the observed

[2]https://www.embedl.com/

performance trends are stable across hardware or partly dependent on the inference stack used in the present study.

Finally, the limited number of completed runs and simulator initialisations restricts the statistical confidence of the present analysis. Future experiments should include more runs across all scenarios to obtain more reliable averages and a clearer view of variation between executions. Useful extensions would also include stricter structured-output validation, comparison with multi-agent reinforcement-learning baselines, and a joint analysis of task performance, inference latency, token usage, and energy cost.

A separate ablation study could examine how the deterministic support around the language model contributes to performance. Relevant variants include following the controller's recommendation without an LLM, presenting the LLM with the complete valid-action set, retaining shaped candidates while removing recommendation fields, and using the complete framework reported in this thesis. Comparing these variants under matched scenarios and seeds would separate the effects of candidate shaping and recommendation information from those of language-model selection. This analysis requires additional controller variants and experiment runs and is therefore outside the scope of the present study.

# References

[1] M. Tomasello, *Origins of Human Communication*. Cambridge, MA, USA: MIT Press, 2008.

[2] R. Dunbar, *Grooming, Gossip, and the Evolution of Language*. Cambridge, MA, USA: Harvard University Press, 1996.

[3] H. H. Clark, *Using Language*. Cambridge, UK: Cambridge University Press, 1996.

[4] S. Johansson, *Origins of Language: Constraints on Hypotheses*, ser. Converging Evidence in Language and Communication Research. Amsterdam, The Netherlands and Philadelphia, PA, USA: John Benjamins Publishing Company, 2005, vol. 5.

[5] S. R. Fischer, *A History of Writing*. London, UK: Reaktion Books, 2004.

[6] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3rd ed., 2026, online manuscript released 6 January 2026. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/

[7] Y. Bengio, R. Ducharme, P. Vincent, and C. Janvin, "A neural probabilistic language model," *J. Mach. Learn. Res.*, vol. 3, pp. 1137–1155, Mar. 2003.

[8] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *Proceedings of the International Conference on Learning Representations Workshop*, 2013. [Online]. Available: https://iclr.cc/archive/2013/workshop-proceedings.html

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[10] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, E. Brynjolfsson, S. Buch, D. Card, R. Castellon, N. Chatterji, A. Chen, K. Creel, J. Q. Davis, D. Demszky, C. Donahue, M. Doumbouya, E. Durmus, S. Ermon, J. Etchemendy, K. Ethayarajh, L. Fei-Fei, C. Finn, T. Gale, L. Gillespie, K. Goel, N. Goodman, S. Grossman,

N. Guha, T. Hashimoto, P. Henderson, J. Hewitt, D. E. Ho, J. Hong, K. Hsu, J. Huang, T. Icard, S. Jain, D. Jurafsky, P. Kalluri, S. Karamcheti, G. Keeling, F. Khani, O. Khattab, P. W. Koh, M. Krass, R. Krishna, R. Kuditipudi, A. Kumar, F. Ladhak, M. Lee, T. Lee, J. Leskovec, I. Levent, X. L. Li, X. Li, T. Ma, A. Malik, C. D. Manning, S. Mirchandani, E. Mitchell, Z. Munyikwa, S. Nair, A. Narayan, D. Narayanan, B. Newman, A. Nie, J. C. Niebles, H. Nilforoshan, J. Nyarko, G. Ogut, L. Orr, I. Papadimitriou, J. S. Park, C. Piech, E. Portelance, C. Potts, A. Raghunathan, R. Reich, H. Ren, F. Rong, Y. Roohani, C. Ruiz, J. Ryan, C. Ré, D. Sadigh, S. Sagawa, K. Santhanam, A. Shih, K. Srinivasan, A. Tamkin, R. Taori, A. W. Thomas, F. Tramèr, R. E. Wang, W. Wang, B. Wu, J. Wu, Y. Wu, S. M. Xie, M. Yasunaga, J. You, M. Zaharia, M. Zhang, T. Zhang, X. Zhang, Y. Zhang, L. Zheng, K. Zhou, and P. Liang, "On the opportunities and risks of foundation models," *arXiv preprint arXiv:2108.07258*, 2021.

[11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[12] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 9118–9147. [Online]. Available: https://proceedings.mlr.press/v162/huang22a.html

[13] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 2609–2634. [Online]. Available: https://aclanthology.org/2023.acl-long.147/

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[15] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models,"

in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract.html

[16] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[17] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.

[18] Z. Yan, N. Jouandeau, and A. A. Cherif, "A survey and analysis of multi-robot coordination," *International Journal of Advanced Robotic Systems*, vol. 10, no. 12, 2013, accessed: 26 February 2026. [Online]. Available: https://journals.sagepub.com/doi/full/10.5772/57313

[19] United Nations Conference on Trade and Development, "Digital economy report 2024," https://unctad.org/publication/digital-economy-report-2024, 2024, accessed: 10 July 2026.

[20] J. J. Bartholdi, III and S. T. Hackman, *Warehouse & Distribution Science*. Atlanta, GA, USA: Georgia Institute of Technology, Aug. 2019, release 0.98.1. [Online]. Available: https://www.warehouse-science.com/book/editions/wh-sci-0.98.1.pdf

[21] R. de Koster, T. Le-Duc, and K. J. Roodbergen, "Design and control of warehouse order picking: A literature review," *European Journal of Operational Research*, vol. 182, no. 2, pp. 481–501, 2007. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0377221706006473

[22] N. Boysen, R. de Koster, and F. Weidinger, "Warehousing in the e-commerce era: A survey," *European Journal of Operational Research*, vol. 277, no. 2, pp. 396–411, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0377221718307185

[23] Ítalo Renan da Costa Barros and T. P. Nascimento, "Robotic mobile fulfillment systems: A survey on recent developments and research opportunities," *Robotics and Autonomous Systems*, vol. 137, p. 103729, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0921889021000142

[24] P. R. Wurman, R. D'Andrea, and M. Mountz, "Coordinating hundreds of cooperative, autonomous vehicles in warehouses," *AI Magazine*, vol. 29, no. 1, pp. 9–20, 2008.

[25] R. Karam, A. A. Nguyen, R. Lin, D. R. Martin, D. Morales, B. A. Butler, and M. Egerstedt, "Collaboration in multi-robot systems: Taxonomy and survey over frameworks for collaboration," 2026. [Online]. Available: https://arxiv.org/abs/2603.23898

[26] G. Papoudakis, F. Christianos, L. Schäfer, and S. V. Albrecht, "Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021. [Online]. Available: https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/a8baa56554f96369ab93e4f3bb068c22-Abstract-round1.html

[27] A. Krnjaic, R. D. Steleac, J. D. Thomas, G. Papoudakis, L. Schäfer, A. W. Keung To, K.-H. Lao, M. Cubuktepe, M. Haley, P. Börsting, and S. V. Albrecht, "Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 677–684.

[28] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016. [Online]. Available: https://arxiv.org/abs/1606.01540

[29] M. Towers, A. Kwiatkowski, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, J. Terry, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," in *Advances in Neural Information Processing Systems*, vol. 38. Curran Associates, Inc., 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/d7ff1795e8527f6443371c3933bdb52b-Abstract-Datasets_and_Benchmarks_Track.html

[30] B. Gerkey and M. Mataric, "A formal analysis and taxonomy of task allocation in multi-robot systems," *I. J. Robotic Res.*, vol. 23, pp. 939–954, 09 2004.

[31] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.

[32] B. Ichter, A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, D. Kalashnikov, S. Levine, Y. Lu, C. Parada, K. Rao, P. Sermanet, A. T. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, M. Yan, N. Brown, M. Ahn, O. Cortes, N. Sievers, C. Tan, S. Xu, D. Reyes, J. Rettinghouse, J. Quiambao, P. Pastor, L. Luu, K.-H. Lee, Y. Kuang, S. Jesmonth, N. J. Joshi, K. Jeffrey, R. J. Ruano, J. Hsu, K. Gopalakrishnan, B. David, A. Zeng, and C. K. Fu, "Do as i can, not as i say: Grounding language in robotic affordances," in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 287–318. [Online]. Available: https://proceedings.mlr.press/v205/ichter23a.html

[33] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9493–9500. [Online]. Available: https://doi.org/10.1109/ICRA48891.2023.10160591

[34] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 286–299, accessed: 26 February 2026. [Online]. Available: https://ieeexplore.ieee.org/document/10610855

[35] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," 2025. [Online]. Available: https://arxiv.org/abs/2402.07927

[36] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen, "Let me speak freely? a study on the impact of format restrictions on large language model performance," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Miami, Florida, USA: Association for Computational Linguistics, 2024, pp. 1218–1236. [Online]. Available: https://aclanthology.org/2024.emnlp-industry.91/

[37] X. Niu and D. G. Broo, "Investigating symbiosis in robotic ecosystems: A case study for multi-robot reinforcement learning reward shaping," in *2025 9th International Conference on Robotics and Automation Sciences (ICRAS)*, 2025, pp. 112–117.

[38] X. Niu, N. C. Barajas, and D. G. Broo, "Enabling symbiosis in multi-robot systems through multi-agent reinforcement learning," in *2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–7.

[39] P. Li, Z. An, S. Abrar, and L. Zhou, "Large language models for multi-robot systems: A survey," *Autonomous Robots*, vol. 50, no. 3, p. 30, 2026. [Online]. Available: https://doi.org/10.1007/s10514-026-10257-4

[40] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1769–1782. [Online]. Available: https://proceedings.mlr.press/v205/huang23c.html

[41] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "Progprompt: Generating situated robot task plans using large language models," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 11 523–11 530. [Online]. Available: https://doi.org/10.1109/ICRA48891.2023.10161317

[42] Z. Liu, W. Yao, J. Zhang, L. Xue, S. Heinecke, R. R. N. Murthy, Y. Feng, Z. Chen, J. C. Niebles, D. Arpit, R. Xu, P. L. Mui, H. Wang, C. Xiong, and S. Savarese, "BOLAA: Benchmarking and orchestrating LLM autonomous agents," in *ICLR 2024 Workshop on Large Language Model Agents*, 2024. [Online]. Available: https://openreview.net/forum?id=BUa5ekiHlQ

[43] V. Bhat, A. U. Kaypak, P. Krishnamurthy, R. Karri, and F. Khorrami, "Grounding large language models for robot task planning using closed-loop state feedback," *Advanced Robotics Research*, Nov. 2025. [Online]. Available: http://dx.doi.org/10.1002/adrr.202500072

[44] A. Dawane, "On-device small language model inference acceleration for real-time robotics: A survey," in *2026 IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC)*, 2026, pp. 669–675.

[45] L. Xiao and T. Yamasaki, "Probing prompt design for socially compliant robot navigation with vision language models," 2026. [Online]. Available: https://arxiv.org/abs/2601.14622

[46] Meta AI, "Llama 3.2: Revolutionizing edge AI and vision with open, customizable models," https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/, 2024, accessed: 10 July 2026.

[47] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. Le Scao, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023. [Online]. Available: https://arxiv.org/abs/2310.06825

[48] Gemma Team, A. Kamath, J. Ferret, S. Pathak, N. Vieillard, R. Merhej, S. Perrin, T. Matejovicova, A. Ramé, M. Rivière *et al.*, "Gemma 3 technical report," *arXiv preprint arXiv:2503.19786*, 2025. [Online]. Available: https://arxiv.org/abs/2503.19786

[49] M. Abdin, J. Aneja, H. Behl, S. Bubeck, R. Eldan, S. Gunasekar, M. Harrison, R. J. Hewett, M. Javaheripi, P. Kauffmann *et al.*, "Phi-4 technical report," *arXiv preprint arXiv:2412.08905*, 2024. [Online]. Available: https://arxiv.org/abs/2412.08905

[50] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024. [Online]. Available: https://arxiv.org/abs/2412.15115

[51] H. Zhao, K. Wang, M. Yu, and H. Mei, "Explicit planning helps language models in logical reasoning," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 11 155–11 173. [Online]. Available: https://aclanthology.org/2023.emnlp-main.688/

[52] K. Valmeekam, M. Marquez, S. Sreedharan, and S. Kambhampati, "On the planning abilities of large language models: A critical investigation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/efb2072a358cefb75886a315a6fcf880-Abstract-Conference.html

# A Appendix

## A.1 Reproducibility: Experiment Runners

The reported experiments are executed from the Battery-TA-RWARE repository using the objective-family runners for centralized and shared-context planning. Earlier exploratory scripts are not part of the reported comparison. The examples below show the command structure used by the current protocol; the seed, prompt format, model subset, and result directory can be changed for a particular session.

```
python Battery-TA-RWARE/scripts/run_obj1_shared_context_llm.py \
  --prompt_format language \
  --max_steps 0 \
  --max_busy_steps_for_replan 10 \
  --max_inactivity_steps 500 \
  --disable_support_needed_soon \
  --find_path_agent_aware_always \
  --seed 0
```

Listing 2: Run Objective 1 with the shared-context planner.

```
python Battery-TA-RWARE/scripts/run_obj1_centralized_llm.py \
  --prompt_format language \
  --max_steps 0 \
  --max_busy_steps_for_replan 10 \
  --max_inactivity_steps 500 \
  --disable_support_needed_soon \
  --find_path_agent_aware_always \
  --seed 0
```

Listing 3: Run Objective 1 with the centralized planner.

```
python Battery-TA-RWARE/scripts/run_obj2_shared_context_llm.py \
  --prompt_format language \
  --max_steps 0 \
  --max_busy_steps_for_replan 10 \
  --max_inactivity_steps 500 \
  --disable_support_needed_soon \
  --find_path_agent_aware_always \
  --seed 0
```

Listing 4: Run Objective 2 with the shared-context planner.

```
python Battery-TA-RWARE/scripts/run_obj2_centralized_llm.py \
  --prompt_format language \
  --max_steps 0 \
  --max_busy_steps_for_replan 10 \
```

```
  --max_inactivity_steps 500 \
  --disable_support_needed_soon \
  --find_path_agent_aware_always \
  --seed 0
```

Listing 5: Run Objective 2 with the centralized planner.

```
python Battery-TA-RWARE/scripts/run_obj3_shared_context_llm.py \
  --prompt_format language \
  --max_steps 0 \
  --max_busy_steps_for_replan 10 \
  --max_inactivity_steps 500 \
  --disable_support_needed_soon \
  --find_path_agent_aware_always \
  --seed 0
```

Listing 6: Run Objective 3 with the shared-context planner.

```
python Battery-TA-RWARE/scripts/run_obj3_centralized_llm.py \
  --prompt_format language \
  --max_steps 0 \
  --max_busy_steps_for_replan 10 \
  --max_inactivity_steps 500 \
  --disable_support_needed_soon \
  --find_path_agent_aware_always \
  --seed 0
```

Listing 7: Run Objective 3 with the centralized planner.

The same commands can be run with --prompt_formatjson to execute the structured prompt variant. Objective 1 also supports --selected_models. For Objectives 2 and 3, the corresponding single-model filter is --only_model. The seed is logged for traceability.

## A.2 Result Directories

The objective-family runners write session outputs under the following default directories:

- results/obj1_shared_context_llm_experiments
- results/obj1_centralized_llm_experiments
- results/obj2_shared_context_llm_experiments
- results/obj2_centralized_llm_experiments

- results/obj3_shared_context_llm_experiments
- results/obj3_centralized_llm_experiments

These raw step-level result directories are retained separately because their size is unsuitable for the source repository; the repository contains the experiment implementation and runner commands. Each session stores aggregate summaries together with per-run JSON summaries and detailed logs. The aggregate analysis uses the retained runs from these outputs and follows the protocol described in Section 3.5.

## A.3 Prompt Excerpt

The full prompts are generated programmatically from the simulator state. The excerpt below is illustrative and shows the action-id grounding and output contract used by the natural-language shared-context prompt.

```
Objective:
Maximize useful warehouse progress under the current objective.

Current agent:
agent_0, role=AGV, position=(8, 3), battery=72, carrying=None

Shared context:
- Requested shelf 24 is at position (11, 5), action id 50.
- Picker support is required at shelf action id 50.
- Charging station action id 7 is currently unoccupied.

Candidate actions:
- action=0, type=NOOP, distance=0, instruction=wait only if blocked
- action=50, type=SHELF, distance=8, instruction=go to requested
    shelf
- action=7, type=CHARGING, distance=11, instruction=use if battery is
    unsafe

Output contract:
Return only:
Action: <integer action id>
Steps: <integer hold duration>
Reason: <short reason>
```

Listing 8: Illustrative shared-context natural-language prompt excerpt.

For the centralized planner, the prompt contains the shared warehouse context plus one planning block for each agent being replanned. The centralized response must contain

one action for each listed agent. For the JSON prompt variant, the same information is represented as a structured object and the model is constrained to return a machine-readable action object or action list.

## A.4 Run Summary Fields

The aggregate CSV files use the following run-level fields:

```
Objective
Arch
Prompt
Model
Scenario
Seed
Run
llm_calls
json_generation_failures
llm_missing_or_invalid_actions
charging_station_utilization_rate
elapsed_seconds
steps to first delivery
total deliveries
total steps
deliveries_per_1000_steps
llm_calls_per_1000_steps
llm_calls_per_delivery
invalid_action_rate
json_failure_rate
summary_json_path
```

Listing 9: Run-summary CSV fields used for aggregation.

The main analysis uses total deliveries, LLM calls, deliveries per 1000 LLM calls, and the descriptive battery-aware productivity score defined in Section 3.5. The remaining fields are retained for filtering, traceability, and diagnostic interpretation. Parsing and invalid-action fields support diagnostic tracing of malformed outputs and configuration errors. The summary_json_path field links each aggregate row to the corresponding detailed run summary.